Trabajo Práctico Final: Clínica

Grupo 6 · Calidad de Software A

Integrantes

Marina Lombardo

Valentino Hernández Pascual

Simón Schleider

Lucas Paniagua

2026

Índice

1. Introducción **2**

1.1. Objetivo del trabajo 2

1.2. Sistema y alcance 2

1.3. Documentación de referencia 3

2. Estrategia y entorno **4**

2.1. Estrategia general y niveles de prueba 4

2.2. Criterio de diseño: Precondiciones vs. Validaciones 6

2.3. Ejecución, correcciones permitidas y fallas forzadas 6

2.4. Entorno, herramientas y convenciones de trazabilidad 7

3. Test unitario (AsociadoDTO) **7**

3.1. Objetivo y alcance 7

3.2. Especificación de referencia y supuestos 8

3.3. Diseño de escenarios 9

3.4. Clases de equivalencia y valores límite 10

3.5. Casos de prueba 11

3.6. Resultados y defectos detectados 12

3.7. Conclusiones del nivel unitario 13

4. Test de integración con MySQL (BaseDeDatos y DataAccessObject) **13**

4.1. Objetivo y alcance 13

4.2. Especificación de referencia y supuestos 13

4.3. Estrategia de integración y estados del entorno 15

4.4. Casos de prueba 16

4.5. Resultados y defectos detectados 18

4.6. Cobertura de fallos exigidos por la consigna y observaciones de código 18

4.7. Conclusiones del nivel de integración 19

5. Testing con Mocks y Concurrencia (Ambulancia, State y RetornoAutomatico) **20**

5.1. Primera parte: Testing atómico con dobles de prueba (Mocks) 20

5.1.1. Objetivo y alcance 20

5.1.2. Especificación de referencia, dobles utilizados y supuestos 21

5.1.3. Clases de equivalencia y casos de prueba (CM1 a CM33) 22

5.1.4. Resultados y defectos detectados con Mocks 24

5.2. Segunda parte: Testing de integración y concurrencia 25

5.2.1. Objetivo, alcance y criterio de validación 25

5.2.2. Recursos compartidos y criterios de determinismo 26

5.2.3. Casos de prueba (CC1 a CC13) 28

5.2.4. Resultados, defectos detectados, falla sembrada y observaciones 29

5.2.5. Conclusiones del Ítem 3 (Mocks y Concurrencia) 30

6. Conclusiones **30**

6.1. Resumen global de ejecución 31

6.2. Registro maestro de defectos 32

6.3. Evaluación metodológica y limitaciones 34

Apéndice A: Resultados detallados de ejecución **36**

A.1. Pruebas unitarias de AsociadoDTO (@Tag("unit")) 36

A.2. Pruebas de integración con MySQL (@Tag("integration")) 36

A.3. Pruebas con Mocks (@Tag("mock")) 38

A.4. Pruebas de integración y concurrencia (@Tag("concurrency")) 41

Apéndice B: Capturas de ejecución en el IDE **43**

B.1. Pruebas unitarias de AsociadoDTO 43

B.2. Pruebas de integración con MySQL 43

B.3. Pruebas con Mocks 44

B.4. Pruebas de integración y concurrencia 44

Apéndice C: Entorno de ejecución y reproducción **46**

C.1. Ajustes del entorno local 46

C.2. Comandos de ejecución por suite 46

Apéndice D: Referencias y trazabilidad con la SRS **48**

1. Introducción

1.1. Objetivo del trabajo

El presente informe documenta la estrategia de testing aplicada al sistema **Clínica**, con el propósito de verificar el cumplimiento de la Especificación de Requisitos de Software (SRS). El diseño de las pruebas se estructura diferenciando dos niveles arquitectónicos:

- **Testing atómico (unitario):** Pruebas sobre componentes independientes ejecutadas en aislamiento. Se aplica al objeto de transferencia de datos AsociadoDTO (Consigna 1) y, mediante el uso de dobles de prueba (*mocks*) que reemplazan a sus colaboradores directos, a la clase Ambulancia y a sus estados concretos (Consigna 3, primera parte).
- **Testing de integración:** Pruebas sobre componentes acoplados entre sí y con recursos externos reales. Se aplica a la capa de persistencia (AsociadoDTO, DataAccessObject y BaseDeDatos) operando contra un motor relacional MySQL real (Consigna 2), y al flujo completo de la máquina de estados acoplada a Ambulancia interactuando con hilos de ejecución concurrentes (Consigna 3, segunda parte).

Los casos de prueba se derivaron a partir de la especificación funcional y se ejecutaron sobre el código entregado por la cátedra, con las únicas adaptaciones descritas en la sección 2.3. Cada prueba fallida se registró como evidencia de la detección de un defecto.

1.2. Sistema y alcance

Clínica es una aplicación de escritorio desarrollada en Java bajo el patrón arquitectónico Modelo-Vista-Controlador (MVC). El alcance de este trabajo abarca dos subsistemas centrales:

1. **Subsistema de persistencia de asociados** (paquete persistencia)**:** Compuesto por AsociadoDTO, encargado de encapsular los atributos del asociado; BaseDeDatos, que administra el ciclo de vida de la conexión JDBC con MySQL aplicando el patrón *Singleton*; y DataAccessObject (DAO), que expone las operaciones CRUD mediante consultas SQL parametrizadas (PreparedStatement).
2. **Subsistema de simulación de ambulancia** (paquete modelo.modeloDominio.ambulancia)**:** La clase Ambulancia actúa como un recurso compartido entre múltiples hilos de ejecución (Asociado, Operario y el hilo independiente RetornoAutomatico). Su comportamiento se rige mediante el patrón *State* a través de la interfaz State, gestionando la exclusión mutua y la sincronización mediante bloques synchronized, wait() y notifyAll(), además de notificar eventos a la vista mediante la interfaz Observable.

Consigna	Componentes bajo prueba	Aspectos verificados
1. Testing Unitario	AsociadoDTO	Asignación y recuperación aislada de atributos en constructores y getters; validaciones internas de integridad en los setters.
2. Integración BD	AsociadoDTO, DataAccessObject, BaseDeDatos (contra MySQL 8)	Instanciación única (Singleton), aprovisionamiento automático del esquema y datos semilla, operaciones CRUD parametrizadas, manejo de excepciones SQL y mapeo de datos.
3. Mocks y Concurrencia	Ambulancia, interfaz State (y estados concretos), RetornoAutomatico	Delegación atómica e interacciones mediante dobles de prueba (Mockito), tabla completa de transición de estados, invariante de ocupación y sincronización entre hilos reales.

Quedan excluidos de las pruebas la interfaz gráfica (VentanaPrincipal, VentanaSimulacion), los controladores y los restantes módulos de dominio (atención médica, internación, facturación y reportes). Asimismo, como regla metodológica del trabajo, el código productivo no se modifica ni corrige en el repositorio, con la única excepción del defecto **D3** en AsociadoDTO, cuya corrección es exigida de forma explícita por el enunciado al iniciar el Ítem 2.

1.3. Documentación de referencia

Para evitar sesgos derivados de la implementación existente, el oráculo de cada caso de prueba se definió exclusivamente a partir de la documentación del proyecto:

Documento	Ubicación	Aporte al diseño de pruebas
Enunciado del TP Final	*docs/Enunciado_Trabajo_final.pdf*	Definición de niveles arquitectónicos, alcance de las suites y criterios de entrega.

Documento	Ubicación	Aporte al diseño de pruebas
SRS de Persistencia	*docs/SRS_Persistencia_Clinica.pdf*	Reglas de validación estructural de AsociadoDTO , requisitos del Singleton e inicialización y operaciones del DAO.
Diagramas UML y Javadoc	Entrega original / Código fuente	Máquina de estados de la ambulancia y contratos de los métodos (pre:, post:, @throws).

2. Estrategia y entorno

2.1. Estrategia general y niveles de prueba

Se adoptó una estrategia primaria de **caja negra** para diseñar los escenarios y casos de prueba (aplicando partición en clases de equivalencia, análisis de valores límite y pruebas de transición de estados) a partir de la SRS, el informe de diseño y los diagramas UML. De manera complementaria, se utilizó inspección de **caja blanca** para analizar la estructura del código productivo, aislar fallos ante desviaciones en las pruebas y documentar anomalías estructurales o riesgos potenciales que no son alcanzables mediante ejecución dinámica.

Nivel de prueba	Componentes	Recursos externos	Técnica principal	Consigna
Unitario (atómico)	AsociadoDTO	Ninguno	Clases de equivalencia y valores límite	1
Integración BD	AsociadoDTO, DataAccessObject, BaseDeDatos	Motor MySQL 8 real	Integración ascendente, estados del entorno y valores representativos	2

Nivel de prueba	Componentes	Recursos externos	Técnica principal	Consigna
Unitario con Mocks	Ambulancia / Estados concretos (State)	Ninguno (Mockito)	Dobles de prueba (espías de interacción) y matriz de transición de estados	3 (Parte A)
Integración y Concurrencia	Ambulancia, estados reales, RetornoAutomatico	Hilos de ejecución reales	Flujos completos de estado y escenarios deterministas de contención	3 (Parte B)

La separación estricta en estas cuatro suites responde a la naturaleza de los defectos que se buscan detectar:

1. **Aislamiento en el nivel unitario (Consigna 1):** AsociadoDTO no posee dependencias hacia otros módulos. Evaluarlo sin servicios externos garantiza que cualquier fallo detectado corresponda exclusivamente a su lógica interna de asignación o validación.
2. **Motor real e integración ascendente (Consigna 2):** Las fallas de sintaxis SQL, las violaciones de restricciones de integridad, los errores de mapeo entre columnas y atributos y los problemas en el ciclo de vida de la conexión JDBC solo se manifiestan al ejecutar las consultas contra un motor de base de datos real; un mock de JDBC devolvería lo que el test le indicara y ocultaría justamente esos defectos. Por eso se probó contra MySQL, con una estrategia de integración ascendente (bottom-up): primero se validó BaseDeDatos sola contra el motor y después DataAccessObject sobre esa capa. Así, cuando un caso del DAO falla, se puede descartar que la causa esté en la conexión o en la creación de la tabla.
3. **Desacoplamiento mediante Mocks (Consigna 3 - Parte A):** Aprovechando la interfaz State, el uso de dobles de prueba permite evaluar por separado la delegación que realiza Ambulancia y las transiciones que solicita cada estado concreto. De esta forma, cada defecto de transición o de notificación quedó ubicado en la clase y el método exactos, sin la interferencia de hilos ni de tiempos de espera.
4. **Integración con estados e hilos reales (Consigna 3 - Parte B):** Los defectos derivados de la interacción entre Ambulancia y sus estados concretos, como la

desactualización del indicador de ocupación tras una transición, no son observables mediante dobles de prueba, ya que dependen del comportamiento conjunto de ambas clases. Del mismo modo, los defectos de sincronización (hilos en espera que no son notificados, esperas dentro de secciones críticas y condiciones de terminación incorrectas en hilos independientes como RetornoAutomatico) solo se manifiestan con múltiples hilos de ejecución reales que acceden concurrentemente al recurso compartido. Por este motivo, se evaluaron los flujos completos de la máquina de estados con las implementaciones concretas y escenarios de contención con hilos reales.

2.2. Criterio de diseño: Precondiciones vs. Validaciones

Para determinar el dominio de entrada de las pruebas, cada restricción documental se clasificó bajo dos categorías:

- **Validaciones del componente (comportamiento verificable):** Cuando la consigna, la SRS o el informe de diseño especifican que el método debe validar activamente el dato de entrada (por ejemplo, las validaciones de integridad en los setters de AsociadoDTO o la validación de no nulidad en DAO.guardar), las entradas fuera de regla conforman **clases de equivalencia inválidas** y se prueban verificando que el componente rechace la operación sin corromper su estado.
- **Precondiciones de contrato (pre:):** Cuando una restricción figura únicamente como una precondición en el contrato Javadoc y la especificación no le asigna al método la responsabilidad de validarla (por ejemplo, el constructor parametrizado de AsociadoDTO o cargar(id) con pre: id > 0), es responsabilidad del cliente garantizar su cumplimiento, por lo que las entradas que violan la precondición carecen de resultado esperado definido y no se prueban.

2.3. Ejecución, correcciones permitidas y fallas forzadas

1. **Ejecución y confirmación:** Cada suite se ejecuta una vez sobre el código provisto. Cada caso que falla se verifica contra el código fuente para confirmar que se debe a un defecto real y no a un error de la prueba, y se registra en el Registro de defectos.
2. **Política de corrección:** El único defecto corregido es D3 (AsociadoDTO.setNumero, commit 6cda0de), porque la Consigna 2 pide "corregir si es necesario el código del ítem 1". Por eso, la suite unitaria se ejecuta sobre el código original y las demás, con D3 corregido. Los defectos de compilación D1 y D2 no se corrigen en el repositorio: las ejecuciones se realizan sobre copias temporales con esos ajustes aplicados (Apéndice C).
3. **Fallas forzadas y sembradas:** Como el código productivo no se modifica, la capacidad de detección se demuestra además desde las propias pruebas: en integración se lleva la base de datos a estados anómalos (CI2, CI6, CI21 y CI22), y en

concurrencia se usa una subclase de prueba sin notifyAll() (falla sembrada FC1, detectada por CC13).

2.4. Entorno, herramientas y convenciones de trazabilidad

Herramienta	Versión	Propósito en el proyecto
Java (OpenJDK)	17.0.21	Ejecución de la aplicación y compilación de las suites de prueba.
Maven + Surefire	3.9 / 3.2.5	Gestión de dependencias y ejecución de tests con la flag -ea (*enable assertions*) activa.
JUnit Jupiter	5.10.2	Framework de testing; segmentación de las 4 suites mediante anotaciones @Tag.
Mockito	5.11.0	Creación de mocks y verificación de interacciones en la Consigna 3.
MySQL Server / Connector-J	8.0 / 8.3.0	Motor relacional real configurado con el usuario progra_c (mysql_native_password).

Para mantener trazabilidad completa entre el diseño del informe, el código fuente de los tests, los reportes de Maven (Apéndice A) y las capturas de pantalla en IntelliJ IDEA (Apéndice B), se utilizó la siguiente nomenclatura estandarizada tanto en las tablas como en los nombres de los métodos y anotaciones @DisplayName:

- **Vn / In:** Clase de equivalencia válida / inválida.
- **En:** Escenario de prueba (estado inicial de los objetos).
- **Sn:** Supuesto adoptado ante vacíos de especificación.
- **CUn / CIn / CMn / CCn:** Caso de prueba Unitario / Integración BD / Mocks / Concurrencia.
- **Dn / FCn:** Defecto detectado en código productivo / Falla sembrada en doble de prueba.

3. Test unitario (AsociadoDTO)

3.1. Objetivo y alcance

Desarrollar y ejecutar una suite de pruebas unitarias que verifique de forma estrictamente aislada la correcta asignación y recuperación de atributos en persistencia.AsociadoDTO, así como las validaciones internas de integridad

implementadas en sus setters, demostrando la detección de defectos sin levantar servicios externos.

- **Alcance:** Constructor por defecto, constructor parametrizado, los siete métodos *setter* y los siete métodos *getter*. La restricción de unicidad del atributo dni corresponde a la capa de persistencia y se evalúa en el nivel de integración.
- **Suite implementada:** src/test/java/item1/AsociadoDTOTest.java, identificada con @Tag("unit").

3.2. Especificación de referencia y supuestos

El criterio de validación parte de la **SRS ("Requisitos de Transferencia de Datos e Integridad")**:

Atributo	Tipo de dato	Regla de validación (SRS 3.1)
nombre, apellido, dni, ciudad, calle, telefono	String	No puede ser nulo (!= null).
numero	int	Debe ser mayor o igual a 0 (>= 0).

Análisis de contratos (Setters vs. Constructor):

- **Validaciones en los Setters:** Se definen tanto clases válidas como clases inválidas, esperando ante una entrada inválida el lanzamiento de un AssertionError y verificando que los atributos previos del DTO permanezcan inalterados.
- **Precondiciones en el Constructor:** Ni el Javadoc ni el Informe de Diseño le asignan validaciones activas al constructor parametrizado. Para este método, las reglas de la SRS operan estrictamente como precondiciones de Diseño por Contrato garantizadas por el llamador, por lo que solo se particiona y prueba su dominio válido.

Supuestos adoptados:

ID	Supuesto	Justificación
S1	Las cadenas vacías ("") y en blanco (" ") pertenecen a la clase válida de cada String.	La SRS únicamente prohíbe null; según el Informe de Diseño, el filtrado de textos vacíos es responsabilidad de la vista (VentanaPrincipal). Se evalúan como valores límite de longitud.
S2	La unicidad del dni no es validada por una instancia aislada de AsociadoDTO.	Un objeto de transferencia en memoria no conoce al resto de las entidades; la restricción UNIQUE se delega al motor MySQL.
S3	El constructor por defecto inicializa los atributos con los valores estándar de Java (null y 0).	Su contrato Javadoc establece como postcondición devolver *"una instancia a inicializar"*.
S15	El fallo de validación en un setter lanza AssertionError sin alterar el estado del objeto.	El Informe de Diseño especifica el uso de assert y la suite se ejecuta con la opción -ea activa en la JVM.

Observaciones de revisión documental y de código:

1. El comentario Javadoc de AsociadoDTO hace referencia a un atributo id, pero la clase carece de dicho campo, así como de su getter y setter.
2. La SRS no especifica restricciones de formato ni longitud para dni o telefono (permitiendo valores alfabéticos como "abc"), por lo que se sugiere refinar la especificación en futuras versiones.
3. Existe una asimetría de diseño entre el constructor parametrizado (que acepta nulos y negativos sin lanzar error) y los setters (que los rechazan mediante assert), haciendo que la integridad del DTO dependa de la vía utilizada para instanciarlo.

3.3. Diseño de escenarios

ID	Descripción del escenario	Estado inicial del objeto
E1	Instancia de AsociadoDTO creada mediante el constructor vacío.	Atributos String en null y numero = 0 (según S3).

ID	Descripción del escenario	Estado inicial del objeto
E2	Instancia de AsociadoDTO inicializada en @BeforeEach con valores válidos y todos distintos entre sí: "Marina", "Lombardo", "40000000", "Mar del Plata", "Ratiery", 777, "2236767676".	Cumple todas las reglas de la SRS 3.1.

El uso de valores diferentes para cada atributo en **E2** garantiza la detección de errores de asignación cruzada (por ejemplo, si setCalle sobrescribiera por error el atributo ciudad). Tras cada operación se invocan los siete getters mediante un método auxiliar (assertDatos) para comprobar tanto el atributo objetivo como la invariancia de los seis restantes.

3.4. Clases de equivalencia y valores límite

Dado que el constructor parametrizado y cada setter constituyen unidades de código independientes, se definen particiones para cada método:

Método	Entrada	Condición (SRS 3.1)	Naturaleza	Clase válida	Clase inválida	Valores límite seleccionados
Constructor	nombre	No nulo	Precondición	**V1:** != null	*(Precondición)*	—
Constructor	apellido	No nulo	Precondición	**V2:** != null	*(Precondición)*	—
Constructor	dni	No nulo	Precondición	**V3:** != null	*(Precondición)*	—
Constructor	ciudad	No nula	Precondición	**V4:** != null	*(Precondición)*	—
Constructor	calle	No nula	Precondición	**V5:** != null	*(Precondición)*	—
Constructor	telefono	No nulo	Precondición	**V6:** != null	*(Precondición)*	—
Constructor	numero	>= 0	Precondición	**V7:** numero >= 0	*(Precondición)*	0
setNombre	nombre	No nulo	Validación	**V8:** != null	**I1:** null	"", " ", null
setApellido	apellido	No nulo	Validación	**V9:** != null	**I2:** null	"", " ", null
setDni	dni	No nulo	Validación	**V10:** != null	**I3:** null	"", " ", null
setCiudad	ciudad	No nula	Validación	**V11:** != null	**I4:** null	"", " ", null
setCalle	calle	No nula	Validación	**V12:** != null	**I5:** null	"", " ", null
setTelefono	telefono	No nulo	Validación	**V13:** != null	**I6:** null	"", " ", null

Método	Entrada	Condición (SRS 3.1)	Naturaleza	Clase válida	Clase inválida	Valores límite seleccionados
setNumero	numero	>= 0	Validación	**V14:** numero >= 0	**I7:** numero < 0	-1, 0, 1, Integer.MAX_VALUE

Justificación de los valores límite:

- **Atributo numero en setNumero:** Se seleccionan el límite inválido (-1), el límite válido inferior (0) y la cota superior del tipo de dato (Integer.MAX_VALUE), permitiendo capturar errores típicos de operador relacional (uso de > en lugar de >=).
- **Atributos String en los setters:** Al no existir cota de longitud en la SRS, se evalúa el límite inferior de longitud ("", longitud 0) y cadenas compuestas por espacios (" "), verificando que los setters no apliquen restricciones excesivas no especificadas (como !isEmpty() o !isBlank()).

3.5. Casos de prueba

Los casos válidos agrupan múltiples clases válidas compatibles, mientras que los casos inválidos evalúan una única entrada inválida por test para evitar que una excepción temprana enmascare el resto de las verificaciones:

ID	Escenario	Acción / Entrada	Clases cubiertas	Resultado esperado	Resultado
CU1	E1	new AsociadoDTO()	—	Getters devuelven null en los String y numero = 0 (S3).	✅ Pasa
CU2	—	Constructor parametrizado con datos válidos y numero = 0	V1 a V7 (límite 0)	Cada getter devuelve exactamente el argumento recibido.	✅ Pasa
CU3	E2	Invocación de los 6 setters String con cadenas válidas nuevas	V8 a V13	Los 6 campos actualizan su valor; numero permanece en 777.	✅ Pasa
CU4	E2	setNumero(0)	V14 (límite 0)	getNumero() devuelve 0; los 6 String no cambian.	❌ **Falla (D3)**
CU5	E2	setNumero(1)	V14 (adyacente 1)	getNumero() devuelve 1; el resto de los campos sin cambios.	✅ Pasa
CU6	E2	setNumero(Integer.MAX_VALUE)	V14 (extremo tipo)	getNumero() devuelve Integer.MAX_VALUE; resto sin cambios.	✅ Pasa

ID	Escenario	Acción / Entrada	Clases cubiertas	Resultado esperado	Resultado
CU7	E2	Los 6 setters String con cadena vacía ""	V8 a V13 (límite "", S1)	Los 6 campos quedan en ""; numero permanece en 777.	✅ Pasa
CU8	E2	Los 6 setters String con espacios " "	V8 a V13 (" ", S1)	Los 6 campos quedan en " "; numero permanece en 777.	✅ Pasa
CU9	E2	setNombre(null)	I1	Lanza AssertionError; estado del DTO idéntico a E2.	✅ Pasa
CU10	E2	setApellido(null)	I2	Lanza AssertionError; estado del DTO idéntico a E2.	✅ Pasa
CU11	E2	setDni(null)	I3	Lanza AssertionError; estado del DTO idéntico a E2.	✅ Pasa
CU12	E2	setCiudad(null)	I4	Lanza AssertionError; estado del DTO idéntico a E2.	✅ Pasa
CU13	E2	setCalle(null)	I5	Lanza AssertionError; estado del DTO idéntico a E2.	✅ Pasa
CU14	E2	setTelefono(null)	I6	Lanza AssertionError; estado del DTO idéntico a E2.	✅ Pasa
CU15	E2	setNumero(-1)	I7 (límite -1)	Lanza AssertionError; estado del DTO idéntico a E2.	✅ Pasa

3.6. Resultados y defectos detectados

La suite unitaria se ejecutó sobre la clase AsociadoDTO (ver detalle de reportes en Apéndice A y captura de ejecución en Apéndice B, Figura B.1):

- **Total de casos ejecutados:** 15 | **Exitosos (Verde):** 14 | **Fallidos (Rojo):** 1 (CU4).
- **Defecto detectado — D3 (AsociadoDTO.setNumero, Severidad Media):** Al invocar setNumero(0) en el caso **CU4**, el método lanza un AssertionError en lugar de asignar el valor 0. Al inspeccionar el código fuente de AsociadoDTO, se comprobó que la aserción está implementada como assert numero > 0 en vez de assert numero >= 0, violando el requisito de la SRS 3.1 (*"Debe ser mayor o igual a 0"*). Cumpliendo con la consigna 2, este defecto fue corregido en el commit 6cda0de, tras lo cual los 15 casos de la suite unitaria pasan en su totalidad.

3.7. Conclusiones del nivel unitario

- La suite diseñada permitió cubrir la totalidad de las 14 clases de equivalencia válidas y las 7 clases inválidas del componente mediante 15 casos de prueba atómicos, verificando tanto la asignación como la preservación del estado ante entradas rechazadas.
- El hallazgo del defecto **D3** demuestra la importancia del análisis de valores límite en ambos lados de la frontera, el defecto de codificación (> por >=) solo quedó expuesto al ejercitar exactamente el punto de frontera 0 en CU4.

4. Test de integración con MySQL (BaseDeDatos y DataAccessObject)

4.1. Objetivo y alcance

Verificar la comunicación e integración real entre AsociadoDTO, DataAccessObject, BaseDeDatos y el motor relacional MySQL 8 (Consigna 2): aplicación del patrón *Singleton* en el manejador de conexión, creación e inicialización automática del esquema (Grupo_7), de la tabla (Asociados) y de los registros semilla (iniciarBD), y ejecución de las operaciones CRUD mediante consultas parametrizadas. Asimismo, demostrar cómo la suite captura excepciones de SQL, errores de mapeo e inconsistencias en el código provisto.

- **Exclusiones de alcance:** La vista y el controlador, dado que únicamente invocan los servicios expuestos por el DAO.
- **Suites implementadas (@Tag("integration"), paquete item2):**
    - DatabaseIntegrationTest: Verifica el patrón Singleton, la creación del esquema, la estructura de la tabla y los datos semilla (CI1 a CI6).
    - DataAccessObjectIntegrationTest: Verifica las operaciones CRUD, el uso de consultas con parámetros, las excepciones de SQL y el mapeo de datos (CI7 a CI23).
    - support/DatabaseTestSupport: Clase auxiliar que administra una conexión JDBC independiente para verificaciones y el reinicio del Singleton entre pruebas.

4.2. Especificación de referencia y supuestos

El criterio de validación se fundamenta en los requisitos de la **SRS 3.2 y 3.3**:

- **Administración de base de datos (SRS 3.2):** BaseDeDatos debe restringir su instanciación a un único manejador activo (*Singleton*), inicializar la conexión con parámetros predefinidos, ejecutar automáticamente iniciarBD() al instanciarse

(creando el esquema objetivo, recreando la tabla Asociados e insertando los registros iniciales de prueba) y proveer interfaces con PreparedStatement.

- **Operaciones de acceso a datos (SRS 3.3):** DataAccessObject debe aceptar una entidad Asociado al guardar, validar que no sea nula, convertirla a AsociadoDTO e insertarla en la base; proveer cargas individuales por identificador único, por lotes acotados (limite) y de colección completa; y actualizar o eliminar registros según criterios únicos como el DNI. Se integran además las reglas de integridad de la SRS 3.1 y los contratos Javadoc (pre:, post: y @throws SQLException).

Asignación de responsabilidades y criterio sobre precondiciones:

- Los requisitos de la SRS 3.2 se prueban de manera directa sobre BaseDeDatos, mientras que el CRUD (SRS 3.3) se ejercita a través de DataAccessObject (lo que indirectamente ejecuta los métodos ejecutarInsert, ejecutarConsulta, ejecutarActualizacion y ejecutarDelete de BaseDeDatos).
- Siguiendo el criterio definido en la Sección 2.2, dado que la SRS 3.3 exige explícitamente que la operación de guardado *"validará que no sea nula"* la entidad recibida, se diseña el caso inválido CI23 (guardar(null)) esperando su rechazo antes de contactar al motor SQL. Por el contrario, las restricciones que solo figuran como pre: en el Javadoc y no son requeridas como validaciones en la SRS (actualizar(null), cargar(0) con pre: id > 0, cargarConLimite(0) con pre: limite > 0 y eliminar con DNI nulo) se tratan como precondiciones de contrato y quedan fuera de prueba.

Supuestos adoptados:

ID	Supuesto	Justificación
S4	cargar(id) con un identificador inexistente devuelve null.	El contrato Javadoc solo especifica el retorno cuando el registro existe.
S5	actualizar y eliminar con un DNI inexistente no lanzan excepción ni alteran filas.	Es el comportamiento estándar de una sentencia SQL UPDATE o DELETE cuya cláusula WHERE no halla coincidencias.
S6	Tras invocar cerrarConexion(), getInstancia() debe entregar una conexión activa y utilizable.	La SRS 3.2 exige mantener *"un único manejador de conexión activo durante toda la ejecución"*.
S7	Las operaciones de carga devuelven los registros ordenados por clave primaria (id).	La SRS no especifica un ordenamiento; el motor InnoDB de MySQL retorna por clave primaria en lecturas secuenciales.

ID	Supuesto	Justificación
S16	La validación de guardar(null) lanza AssertionError y no modifica la tabla.	El Informe de Diseño detalla que el DAO valida mediante assert la no nulidad del objeto antes de contactar a la base de datos.

4.3. Estrategia de integración y estados del entorno

Todos los componentes que intervienen en esta etapa (MySQL 8, BaseDeDatos, DataAccessObject, AsociadoDTO, Asociado y Domicilio) son **instancias reales**, dado que el uso de *mocks* impediría detectar fallos reales de comunicación JDBC, sintaxis SQL y mapeo de columnas. La estrategia se apoya en cuatro pilares:

1. **Integración ascendente (*bottom-up*):** Se valida primero el módulo inferior (BaseDeDatos) contra MySQL y luego el módulo superior (DataAccessObject). Así, si falla una operación del DAO, se descarta que el problema provenga de la conexión o de la creación de la tabla.
2. **Verificación por canal independiente:** Toda operación de escritura (guardar, actualizar, eliminar) se comprueba abriendo una conexión JDBC exclusiva del test y ejecutando un SELECT propio, sin utilizar los métodos de lectura del DAO. Esto evita que errores simétricos de mapeo (por ejemplo, invertir calle y ciudad tanto al insertar como al leer) o pérdidas silenciosas de datos se enmascaren entre sí, además de confirmar que los cambios se hayan persistido efectivamente fuera de la sesión del DAO.
3. **Aislamiento de estado entre pruebas (@BeforeEach):** Antes de cada caso de prueba se reinicia la instancia del Singleton BaseDeDatos mediante reflexión (sin alterar el código productivo) y se vuelve a invocar getInstancia(), disparando iniciarBD() para restaurar la tabla Asociados con sus 20 registros semilla originales.
4. **Datos de prueba diferenciados:** Para las inserciones y actualizaciones se emplean valores únicos en cada atributo ("Valentino", "Hernan", "40111222", "Calle", 777, "Ciudad", "2230001111"), permitiendo detectar cualquier cruce de índices en los parámetros del PreparedStatement.

Flujo de ejecución trazado sobre el código: En la operación guardar(asociado), el DAO invoca internamente CrearDTO(asociado) para instanciar un AsociadoDTO y delega en BaseDeDatos.ejecutarInsert(...), que prepara el INSERT parametrizado y ejecuta executeUpdate() en MySQL. En la operación cargar(id), el DAO delega en BaseDeDatos.ejecutarConsulta(...) pasándole el id; MySQL devuelve el ResultSet, pero el DAO intenta leer las columnas directamente sin invocar antes rs.next(), lo que origina el defecto **D5**.

Estados del entorno y valores representativos:

Elemento / Parámetro	Particiones / Estados evaluados	Mecanismo de generación en la suite
Esquema Grupo_7	Existente / Inexistente	Ejecución normal / Sentencia DROP DATABASE previa en CI2.
Tabla Asociados	Con semilla (20 filas) / Con fila extra / Con numero = 0 / Inexistente	@BeforeEach / INSERT o DROP TABLE vía conexión propia del test.
Manejador Singleton	Recién inicializado / Con conexión cerrada	@BeforeEach / Invocación a cerrarConexion() en CI6.
Parámetro id (cargar)	Primera fila (1), última fila semilla (20), inexistente (21)	Valores frontera del conjunto semilla de 20 registros.
Parámetro limite	Mínimo (1), exacto al total (20), superior al total (21)	Fronteras respecto a la cardinalidad de los datos semilla.
Parámetro dni	Nuevo ("40111222"), duplicado ("12345678"), inyección SQL ("' OR '1'='1")	Datos de prueba del test y DNI perteneciente a la fila semilla 1.
Entidad Asociado	Instancia válida / Referencia null	Método auxiliar asociadoValido() / null en CI23.

4.4. Casos de prueba

ID	Operación y entrada	Resultado esperado	Resultado
CI1	Invocar BaseDeDatos.getInstancia() dos veces consecutivas	Ambas llamadas retornan la misma referencia de objeto.	☒ Pasa
CI2	Eliminar el esquema Grupo_7 e instanciar el Singleton	iniciarBD() recrea automáticamente el esquema Grupo_7.	☒ Pasa
CI3	Inspeccionar catálogo tras la inicialización de BaseDeDatos	Existe la tabla Asociados con id y los 7 atributos del asociado.	☒ Pasa
CI4	Auditar los registros cargados tras la inicialización	Existen 20 filas semilla sin campos nulos, con numero >= 0 y DNIs únicos.	☒ Pasa
CI5	Insertar una fila adicional e invocar iniciarBD()	La tabla se recrea limpia y vuelve a tener exactamente 20 filas.	☒ Pasa

ID	Operación y entrada	Resultado esperado	Resultado
CI6	Invocar cerrarConexion() y solicitar getInstancia()	Devuelve un manejador con una conexión abierta y activa (S6).	❌ **Falla (D4)**
CI7	guardar(asociado) con todos los datos válidos y distintos	Se inserta una nueva fila (total 21) con cada atributo en su columna correcta.	✅ Pasa
CI8	guardar(asociado) con el DNI ya existente en la fila 1 ("12345678")	Lanza SQLIntegrityConstraintViolationException; la tabla conserva 20 filas.	✅ Pasa
CI9	cargar(1) (primer registro de la semilla)	Devuelve un AsociadoDTO cuyos campos coinciden con la fila 1.	❌ **Falla (D5)**
CI10	cargar(20) (último registro de la semilla)	Devuelve un AsociadoDTO cuyos campos coinciden con la fila 20.	❌ **Falla (D5)**
CI11	cargar(21) (identificador inexistente)	Devuelve null sin lanzar excepción (S4).	❌ **Falla (D5)**
CI12	cargarConLimite(1)	Devuelve una colección con 1 único elemento correspondiente a la fila 1.	✅ Pasa
CI13	cargarConLimite(20)	Devuelve una colección con las 20 filas semilla.	✅ Pasa
CI14	cargarConLimite(21) (límite superior a las filas existentes)	Devuelve las 20 filas disponibles sin producir error.	✅ Pasa
CI15	cargarBD()	Devuelve las 20 filas completas mapeadas correctamente a objetos DTO.	✅ Pasa
CI16	actualizar(asociado) correspondiente a la fila 7 (DNI "22334455")	Modifica los datos de la fila 7 y mantiene intactas las 19 filas restantes.	❌ **Falla (D6)**
CI17	actualizar(asociado) con un DNI que no existe en la tabla	No modifica ninguna fila ni lanza excepción (S5).	❌ **Falla (D6)**
CI18	eliminar(dni) de un asociado existente en la semilla	Quedan 19 filas en la tabla y el DNI indicado ya no se encuentra presente.	✅ Pasa
CI19	eliminar(dni) con un DNI inexistente	La tabla permanece intacta con sus 20 filas (S5).	✅ Pasa
CI20	eliminar("' OR '1'='1") (intento de inyección SQL)	El payload se trata como parámetro literal; no se elimina ninguna fila.	✅ Pasa
CI21	Insertar fila con numero = 0 por vía externa e invocar cargarBD()	Retorna 21 DTOs mapeando correctamente el último registro con numero = 0.	✅ Pasa
CI22	Eliminar la tabla Asociados e invocar cargarBD()	Propaga SQLException hacia el llamador según declara el Javadoc.	✅ Pasa
CI23	guardar(null)	Lanza AssertionError antes de ejecutar SQL; la tabla sigue con 20 filas (S16).	✅ Pasa

4.5. Resultados y defectos detectados

La suite se ejecutó contra MySQL 8.0 sobre el código con el defecto **D3** de AsociadoDTO ya corregido (6cda0de), tal como solicita la consigna 2 (ver reportes en Apéndice A y captura en Apéndice B, Figura B.2):

- **Total de casos ejecutados:** 23 | **Exitosos (Verde):** 17 | **Fallidos (Rojo):** 6 (CI6, CI9, CI10, CI11, CI16 y CI17).
- **Defectos detectados en la capa de persistencia:**
    - **D4 (BaseDeDatos.getInstancia, Severidad Media):** Detectado por **CI6**. Luego de invocar cerrarConexion(), las llamadas subsiguientes a getInstancia() siguen devolviendo la misma instancia con su conexión interna cerrada (isClosed() == true), incumpliendo la SRS 3.2 (*"un único manejador de conexión activo"*).
    - **D5 (DataAccessObject.cargar, Severidad Alta):** Detectado por **CI9, CI10 y CI11**. El método intenta extraer los campos del ResultSet devuelto por la consulta individual sin posicionar el cursor mediante rs.next(). En consecuencia, cualquier intento de lectura por ID arroja SQLException: Before start of result set (cuando la fila existe) o Illegal operation on empty result set (cuando no existe), impidiendo por completo la recuperación individual requerida en la SRS 3.3.
    - **D6 (BaseDeDatos.ejecutarActualizacion, Severidad Alta):** Detectado por **CI16 y CI17**. La sentencia UPDATE incluye la condición WHERE id = ?, pero el código fija de manera estática statement.setInt(8, 1), intentando modificar siempre el registro con id = 1 sin importar qué asociado se esté actualizando. Esto viola la SRS 3.3 y provoca dos fallos graves: al actualizar el asociado de la fila 7 (CI16), intenta asignarle a la fila 1 el DNI de la fila 7 y MySQL aborta con Duplicate entry '22334455'; peor aún, al intentar actualizar un DNI inexistente (CI17), sobrescribe silenciosamente los datos de la fila 1 sin lanzar ningún error.

4.6. Cobertura de fallos exigidos por la consigna y observaciones de código

La consigna 2 solicita demostrar específicamente cómo la suite atrapa distintas categorías de problemas en la integración con la base de datos:

Categoría exigida	Mecanismo de comprobación en la suite	Casos asociados
Excepciones de SQL	Verifica que la violación de restricción UNIQUE (DNI duplicado) y la consulta sobre una tabla inexistente propaguen SQLException al llamador.	CI8, CI22 (Pasan)
Errores de mapeo	Detecta la lectura inválida del ResultSet sin next() (**D5**) y comprueba la correspondencia exacta columna-atributo en lecturas masivas, incluyendo el valor frontera numero = 0 una vez corregido **D3**.	CI9–CI11 (Fallan); CI15, CI21 (Pasan)
Inconsistencias lógicas / BD	Detecta la sobrescritura forzada de la fila 1 en el UPDATE (**D6**), la entrega de conexiones cerradas en el Singleton (**D4**) y el rechazo de entidades nulas antes de tocar la base.	CI6, CI16, CI17 (Fallan); CI23 (Pasa)

Observaciones adicionales por inspección de código (caja blanca):

- La rutina iniciarBD() crea la tabla Asociados sin declarar restricciones NOT NULL en sus columnas, y define longitudes (VARCHAR(30), VARCHAR(10) para DNI, etc.) que no están documentadas en la SRS.
- El método CrearDTO(int id, ...) recibe el id leído de la tabla pero lo descarta porque AsociadoDTO no posee dicho atributo.
- ejecutarConsulta no cierra nunca el PreparedStatement creado, generando fuga de recursos.
- abrirConexion e iniciarBD capturan SQLException limitándose a imprimir la traza en consola; si el motor MySQL está caído o rechaza las credenciales, getInstancia() entrega un objeto con la conexión en null que luego falla con NullPointerException.
- La clase de dominio Domicilio exige en su validación numero > 0, entrando en contradicción con la SRS 3.1 que admite numero >= 0.
- BaseDeDatos.getInstancia() carece del modificador synchronized, por lo que no es *thread-safe*.

4.7. Conclusiones del nivel de integración

- La ejecución contra un motor MySQL 8 real permitió descubrir tres defectos críticos (**D4**, **D5** y **D6**) que eran invisibles en las pruebas unitarias, ya que residen en la construcción de los PreparedStatement, el manejo del cursor de ResultSet y el estado de la conexión JDBC.

- El caso **CI17** (defecto **D6**) validó la decisión de auditar las escrituras mediante consultas SQL independientes: dado que actualizar un DNI inexistente sobrescribió la fila 1 sin arrojar excepciones, solo el contraste directo contra el estado completo de la tabla permitió evidenciar la corrupción silenciosa de los datos.
- El caso **CI21** demostró cómo un defecto unitario (**D3**) se propaga hacia la capa de integración como un fallo de mapeo: con el AsociadoDTO original, un registro válido en la base de datos con numero = 0 provocaba la caída de cargarBD() al instanciar los DTOs; tras aplicar la corrección en AsociadoDTO, CI21 pasó sin requerir cambios en el DAO.

5. Testing con Mocks y Concurrencia (Ambulancia, State y RetornoAutomatico)

5.1. Primera parte: Testing atómico con dobles de prueba (Mocks)

5.1.1. Objetivo y alcance

Aprovechar el desacoplamiento arquitectónico provisto por la interfaz State para evaluar de forma atómica y aislada las dos partes que componen el patrón *State* utilizando dobles de prueba (Consigna 3, primera parte):

1. **Prueba aislada de Ambulancia (AmbulanciaMockTest, casos CM1 a CM9):** Se reemplaza la interfaz State por un *mock* (simulando la ausencia temporal de las implementaciones concretas de los estados) y se reemplaza la vista por un mock(Observer.class), verificando que Ambulancia gestione su ocupación, delegue cada evento en el método correspondiente del estado activo y notifique a los observadores.
2. **Prueba aislada de los estados concretos (StatesMockTest, casos CM10 a CM33):** Se instancia cada una de las seis clases concretas de estado inyectándoles un mock(Ambulancia.class), verificando para cada evento si el estado solicita la transición hacia el estado esperado (setState) o si rechaza la solicitud notificando a los observadores (notifyObservers).

Quedan fuera del alcance de esta suite los escenarios donde la ambulancia se encuentra ocupada con la simulación activa al recibir una nueva solicitud, ya que invocan wait() y bloquean el hilo; dichos escenarios se evalúan en la sección 5.2.

5.1.2. Especificación de referencia, dobles utilizados y supuestos

El criterio de validación se extrae del diagrama UML de máquina de estados, los contratos Javadoc de Ambulancia y de sus estados, y el Informe de Diseño, el cual establece que el sistema *"notifica a observadores cuando la ambulancia se encuentra en un estado en el que no puede cumplir una solicitud"*.

Matriz de transición de estados especificada: *(En las celdas donde figura "Rechaza", el estado no debe invocar setState y debe invocar notifyObservers)*:

Estado actual	Evento: Traslado	Evento: Atención domicilio	Evento: Mantenimiento	Evento: Retorno clínica
Disponible	TrasladandoPacienteClinica	AtendiendoEnDomicilio	EnTaller	Sin acción (no cambia)
AtendiendoEnDomicilio	Rechaza	Rechaza	Rechaza	RegresandoClinicaSinPaciente
TrasladandoPacienteClinica	Rechaza	Rechaza	Rechaza	Disponible
EnTaller	Rechaza	Rechaza	RegresandoDelTaller	Rechaza (S9)
RegresandoDelTaller	Rechaza	Rechaza	Rechaza	Disponible
RegresandoClinicaSinPaciente	TrasladandoPacienteClinica	AtendiendoEnDomicilio	Rechaza	Disponible

Dobles de prueba implementados (Mockito 5):

Suite	Doble de prueba	Colaborador reemplazado	Verificación realizada (Espía de interacción)
AmbulanciaMockTest	mock(State.class)	Las 6 clases de estado concreto	Comprobar a qué método de State delega Ambulancia y que lo haga exactamente una vez (verify, verifyNoMoreInteractions).

Suite	Doble de prueba	Colaborador reemplazado	Verificación realizada (Espía de interacción)
AmbulanciaMockTest	mock(Observer.class)	Vista de la simulación	Comprobar que Ambulancia invoque la notificación de cambios.
AmbulanciaMockTest	doAnswer en State	Estado que ejecuta transición	Simular un cambio de estado dinámico al atender un pedido (CM9).
StatesMockTest	mock(Ambulancia.class)	Clase Ambulancia real	Verificar setState(isA(...)) en las transiciones válidas y notifyObservers(...) en los rechazos.

Supuestos adoptados:

- **S8:** En las notificaciones a observadores se verifica que se produzca la invocación con un objeto de notificación.
- **S9:** El evento RetornoClinica sobre EnTallerState se evalúa bajo la regla general de rechazo con notificación (CM25), ya que el Javadoc indica que desde el taller *"no puede retornar a la clínica"*.

5.1.3. Clases de equivalencia y casos de prueba (CM1 a CM33)

Para AmbulanciaMockTest se particionan las variables de **Ocupación** (libre / ocupada), **Estado de la simulación** (activa / terminada) y **Respuesta del mock de estado** (sin transición / cambia de estado). Para StatesMockTest se aplica la técnica de prueba de transición de estados cubriendo las **24 celdas** de la matriz de transiciones:

Casos de AmbulanciaMockTest (con State mockeado):

ID	Estado inicial	Acción invocada	Resultado esperado	Resultado
CM1	Libre, sim. activa	atenderDomicilio(a)	Delega SolicitudDeAtencionDomicilio 1 vez, pasa a ocupada y notifica.	✅ Pasa
CM2	Libre, sim. activa	trasladarALaClinica(a)	Delega SolicitudDeTraslado 1 vez, pasa a ocupada y notifica.	✅ Pasa
CM3	Libre, sim. activa	solicitarMantenimiento(o)	Delega SolicitudMantenimiento 1 vez, pasa a ocupada y notifica.	✅ Pasa

ID	Estado inicial	Acción invocada	Resultado esperado	Resultado
CM4	Ocupada, sim. activa	regresarSinPaciente()	Delega RetornoClinica 1 vez, libera ocupación y notifica.	✅ Pasa
CM5	Ocupada, sim. activa	volviendoDelTaller()	Delega SolicitudMantenimiento 1 vez, libera ocupación y notifica.	✅ Pasa
CM6	Libre, sim. activa	retornoAutomatico(r)	Delega RetornoClinica 1 vez en el estado actual.	✅ Pasa
CM7	Libre, sim. terminada	atenderDomicilio(a)	Retorna inmediatamente sin delegar en el estado ni marcarse ocupada.	✅ Pasa
CM8	Ocupada, sim. terminada	trasladarALaClinica(a)	No se bloquea en wait() (retorna en < 2 s) y no delega en el estado.	✅ Pasa
CM9	Mock de estado configurado para cambiar a un 2° mock	trasladarALaClinica(a) seguido de regresarSinPaciente()	La segunda llamada delega RetornoClinica en el nuevo estado y no en el inicial.	✅ Pasa

Casos de StatesMockTest (con Ambulancia mockeada):

ID	Estado concreto bajo prueba	Evento disparado	Interacción esperada sobre el mock de Ambulancia	Resultado
CM10	DisponibleState	Traslado	setState(TrasladandoPacienteClinicaState)	✅ Pasa
CM11	DisponibleState	Atención domicilio	setState(AtendiendoEnDomicilioState)	✅ Pasa
CM12	DisponibleState	Mantenimiento	setState(EnTallerState)	✅ Pasa
CM13	DisponibleState	Retorno clínica	Sin cambio de estado (setState no invocado)	✅ Pasa
CM14	AtendiendoEnDomicilioState	Traslado	Rechaza (notifyObservers, sin setState)	✅ Pasa
CM15	AtendiendoEnDomicilioState	Atención domicilio	Rechaza (notifyObservers, sin setState)	❌ **Falla (D7)**
CM16	AtendiendoEnDomicilioState	Mantenimiento	Rechaza (notifyObservers, sin setState)	✅ Pasa
CM17	AtendiendoEnDomicilioState	Retorno clínica	setState(RegresandoClinicaSinPacienteState)	✅ Pasa
CM18	TrasladandoPacienteClinicaState	Traslado	Rechaza (notifyObservers, sin setState)	✅ Pasa

ID	Estado concreto bajo prueba	Evento disparado	Interacción esperada sobre el mock de Ambulancia	Resultado
CM19	TrasladandoPacienteClinicaState	Atención domicilio	Rechaza (notifyObservers, sin setState)	❌ **Falla (D7)**
CM20	TrasladandoPacienteClinicaState	Mantenimiento	Rechaza (notifyObservers, sin setState)	✅ Pasa
CM21	TrasladandoPacienteClinicaState	Retorno clínica	setState(DisponibleState)	❌ **Falla (D8)**
CM22	EnTallerState	Traslado	Rechaza (notifyObservers, sin setState)	✅ Pasa
CM23	EnTallerState	Atención domicilio	Rechaza (notifyObservers, sin setState)	❌ **Falla (D7)**
CM24	EnTallerState	Mantenimiento	setState(RegresandoDelTallerState)	✅ Pasa
CM25	EnTallerState	Retorno clínica	Rechaza (notifyObservers, sin setState, S9)	❌ **Falla (D7)**
CM26	RegresandoDelTallerState	Traslado	Rechaza (notifyObservers, sin setState)	✅ Pasa
CM27	RegresandoDelTallerState	Atención domicilio	Rechaza (notifyObservers, sin setState)	❌ **Falla (D7)**
CM28	RegresandoDelTallerState	Mantenimiento	Rechaza (notifyObservers, sin setState)	✅ Pasa
CM29	RegresandoDelTallerState	Retorno clínica	setState(DisponibleState)	✅ Pasa
CM30	RegresandoClinicaSinPacienteState	Traslado	setState(TrasladandoPacienteClinicaState)	✅ Pasa
CM31	RegresandoClinicaSinPacienteState	Atención domicilio	setState(AtendiendoEnDomicilioState)	✅ Pasa
CM32	RegresandoClinicaSinPacienteState	Mantenimiento	Rechaza (notifyObservers, sin setState)	✅ Pasa
CM33	RegresandoClinicaSinPacienteState	Retorno clínica	setState(DisponibleState)	✅ Pasa

5.1.4. Resultados y defectos detectados con Mocks

La ejecución de la suite @Tag("mock") arrojó los siguientes resultados (ver Apéndice A y captura en Apéndice B, Figura B.3):

- **AmbulanciaMockTest:** 9 casos ejecutados | 9 exitosos | 0 fallidos.
- **StatesMockTest:** 24 casos ejecutados | 18 exitosos | 6 fallidos (CM15, CM19, CM21, CM23, CM25, CM27).

- **Total de la suite de Mocks:** 33 casos | **Exitosos (Verde):** 27 | **Fallidos (Rojo):** 6.

Defectos detectados en los estados concretos:

- **D7 (Falta de notificación en rechazos, Severidad Baja):** Detectado por **CM15, CM19, CM23, CM25 y CM27**. En el método SolicitudDeAtencionDomicilio de cuatro estados (AtendiendoEnDomicilio, TrasladandoPacienteClinica, EnTaller y RegresandoDelTaller), así como en EnTallerState.RetornoClinica, la llamada a ambulancia.notifyObservers(...) se encuentra comentada en el código fuente, incumpliendo el Informe de Diseño.

- **D8 (TrasladandoPacienteClinicaState.RetornoClinica, Severidad Media):** Detectado por **CM21**. Al recibir el evento de retorno a la clínica mientras traslada a un paciente, el estado invoca setState(new RegresandoDelTallerState(...)) en lugar de pasar a DisponibleState como especifican tanto el diagrama UML como su propio comentario Javadoc.

5.2. Segunda parte: Testing de integración y concurrencia

5.2.1. Objetivo, alcance y criterio de validación

Evaluar el flujo completo de la máquina de estados real acoplada a la clase Ambulancia, validando la correcta gestión del recurso compartido entre hilos de ejecución independientes, la exclusión mutua en bloques synchronized, la coordinación mediante wait() / notifyAll() y el funcionamiento del hilo periódico RetornoAutomatico (Consigna 3, segunda parte).

- **Suites implementadas (@Tag("concurrency"), paquete item3, sin uso de mocks):**
  - AmbulanciaStateMachineTest (CC1 a CC5): Evalúa secuencias completas de transiciones acoplando Ambulancia con las seis clases de estado reales sobre un único hilo.
  - AmbulanciaConcurrencyTest (CC6 a CC13): Evalúa escenarios de contención, espera y despertar con múltiples hilos concurrentes y el hilo RetornoAutomatico.

Invariante de ocupación y supuestos: A partir de los contratos Javadoc de los constructores de cada estado concreto, se deduce el **invariante de ocupación** que se verifica tras cada transición: el método isOcupado() debe devolver true si y solo si el estado actual es AtendiendoEnDomicilioState, TrasladandoPacienteClinicaState o EnTallerState, y false en los tres estados restantes (Disponible, RegresandoClinicaSinPaciente y RegresandoDelTaller). Si este indicador queda desfasado respecto al estado real, los hilos clientes quedan bloqueados indefinidamente en wait() o ingresan simultáneamente violando la lógica del dominio.

ID	Supuesto	Justificación
S10	Tras ejecutarse retornoAutomatico, debe preservarse el invariante de ocupación.	Si la transición hacia un estado libre no actualiza ocupado = false, los hilos en espera jamás pueden tomar la ambulancia.
S11	Una solicitud rechazada por el estado actual no debe dejar la ambulancia marcada como ocupada.	Se desprende del invariante de ocupación; resuelve una ambigüedad del Javadoc de Ambulancia que indica *"se ocupa, delega"* sin contemplar el caso en que el estado rechace el pedido.
S12	Ningún hilo debe ejecutar esperas pasivas (Thread.sleep) reteniendo el monitor (synchronized) de Ambulancia.	Principio fundamental de concurrencia: dormir dentro de un método sincronizado bloquea incluso las simples consultas de estado (isOcupado()) de otros hilos.
S13	Una simulación reducida (2 asociados con 1 pedido cada uno y 1 operario) finaliza en menos de 2 minutos.	Cota superior calculada en base a las pausas aleatorias de TiempoMuerto (hasta 8 segundos por iteración).
S14	Al finalizar la simulación (setSimulacionActiva(false)), el hilo RetornoAutomatico termina su ejecución en un plazo máximo de 10 segundos.	El Javadoc establece que el hilo cicla *"mientras la simulación esté activa"*.

5.2.2. Recursos compartidos y criterios de determinismo

El análisis de sincronización sobre la clase Ambulancia identificó el siguiente esquema de protección de sus atributos compartidos:

Recurso compartido	Hilos que acceden	Mecanismo de protección en el código
estadoActual	Asociados, Operario, RetornoAutomatico	setState es synchronized, pero getEstadoActual no lo es y el atributo carece del modificador volatile.
ocupado	Todos los hilos de simulación	Métodos getter (isOcupado) y setter (setOcupado) declarados synchronized.
isSimulacionActiva	Todos los hilos y Clinica.finalizarSimulacion	Métodos getter y setter declarados synchronized.
Monitor de Ambulancia	Hilos de pedidos y de liberación	wait() dentro de bucles while en los pedidos; notifyAll() en las liberaciones y al finalizar la simulación.

Para garantizar que los resultados de la suite concurrente sean **deterministas y reproducibles** (evitando falsos positivos o negativos por carga del procesador), se aplicaron las siguientes técnicas de diseño:

- **Sincronización por estados observables del hilo:** Ningún test utiliza esperas fijas arbitrarias para coordinar hilos; se aguarda activamente hasta que el hilo secundario alcance el estado Thread.State.WAITING (confirmando que ingresó en wait()) o se produzca el cambio de estado esperado, con plazos máximos holgados (5 s para despertares y 15 s para el retorno automático).
- **Comprobación de bloqueo sin depender del reloj (CC11):** En lugar de medir milisegundos de demora (lo cual generó un falso positivo inicial bajo alta carga de CPU), CC11 comprueba que isOcupado() responda mientras el hilo que ejecuta la transición lenta permanece simultáneamente en estado TIMED_WAITING dentro de su Thread.sleep.
- **Hilos *daemon* e instancias limpias:** Cada caso crea una instancia nueva de Ambulancia y lanza hilos marcados como *daemon*, asegurando que un hilo bloqueado por un defecto del código no cuelgue la ejecución de Maven ni contamine la prueba siguiente.

5.2.3. Casos de prueba (CC1 a CC13)

ID	Escenario e hilos intervinientes	Secuencia de acciones	Resultado esperado	Resultado
CC1	1 hilo (flujo completo de domicilio)	atenderDomicilio → regresarSinPaciente → retornoAutomatico	Pasa por AtendiendoEnDomicilio (ocupada), RegresandoClinicaSinPaciente (libre) y termina en Disponible (libre).	✅ Pasa
CC2	1 hilo (flujo completo de traslado)	trasladarALaClinica → regresarSinPaciente	Pasa a TrasladandoPacienteClinica (ocupada) y al regresar queda en Disponible (libre).	❌ **Falla (D8)**
CC3	1 hilo (flujo completo de taller)	solicitarMantenimiento → volviendoDelTaller → retornoAutomatico	Pasa por EnTaller (ocupada), RegresandoDelTaller (libre) y termina en Disponible (libre).	✅ Pasa
CC4	1 hilo (retorno automático directo)	atenderDomicilio → retornoAutomatico	Queda en estado RegresandoClinicaSinPaciente y libre (isOcupado() == false, S10).	❌ **Falla (D9)**
CC5	1 hilo (rechazo en estado libre)	solicitarMantenimiento → volviendoDelTaller → atenderDomicilio	El pedido de domicilio es rechazado; permanece en RegresandoDelTaller y libre (isOcupado() == false, S11).	❌ **Falla (D10)**
CC6	Hilo principal ocupa en domicilio; 2° hilo solicita traslado	Esperar 2° hilo en WAITING; invocar regresarSinPaciente()	Mientras espera, el estado no cambia; al liberar, el 2° hilo despierta y pasa a TrasladandoPacienteClinica.	✅ Pasa
CC7	Hilo principal ocupa en taller; 2° hilo solicita atención	Esperar 2° hilo en WAITING; invocar volviendoDelTaller()	Al liberar el taller, el hilo en espera se despierta y es atendido.	✅ Pasa
CC8	2 hilos esperando ambulancia ocupada	Invocar setSimulacionActiva(false)	Ambos hilos se despiertan y finalizan sin delegar en el estado.	✅ Pasa
CC9	Hilo RetornoAutomatico real con simulación activa	Ambulancia en RegresandoClinicaSinPaciente; esperar hasta 15 s	El hilo ejecuta el retorno periódico y devuelve la ambulancia a DisponibleState.	❌ **Falla (D11)**
CC10	Hilo RetornoAutomatico con simulación inactiva	Iniciar hilo con simulacionActiva = false; esperar hasta 15 s (10 s de S14 más 5 s de margen)	El hilo termina su ejecución sin cambiar el estado a Disponible (S14).	✅ Pasa

ID	Escenario e hilos intervinientes	Secuencia de acciones	Resultado esperado	Resultado
CC11	1 hilo ejecuta retorno desde Trasladando; 2° hilo consulta isOcupado()	Invocar isOcupado() mientras el 1° hilo está dentro del retorno	La consulta responde sin quedarse bloqueada esperando que termine el sleep del retorno (S12).	❌ **Falla (D12)**
CC12	Simulación integrada (2 Asociado, 1 Operario, RetornoAutomatico)	Iniciar todos los hilos reales de dominio y aguardar finalización	Todos los hilos completan sus solicitudes y terminan sin *deadlock* en < 2 min (S13).	✅ Pasa
CC13	Falla sembrada **FC1** (AmbulanciaWithoutNotifyAll)	Ejecutar el mismo escenario de espera y liberación de CC6	La prueba detecta que el hilo en espera no se despierta por la ausencia de notifyAll().	✅ Pasa (Detecta FC1)

5.2.4. Resultados, defectos detectados, falla sembrada y observaciones

La ejecución de la suite @Tag("concurrency") arrojó los siguientes resultados (ver Apéndice A y captura en Apéndice B, Figura B.4):

- **AmbulanciaStateMachineTest:** 5 casos ejecutados | 2 exitosos | 3 fallidos (CC2, CC4, CC5).
- **AmbulanciaConcurrencyTest:** 8 casos ejecutados | 6 exitosos | 2 fallidos (CC9, CC11).
- **Total de la suite de Concurrencia:** 13 casos | **Exitosos (Verde):** 8 | **Fallidos (Rojo):** 5.

Defectos detectados en la máquina de estados y concurrencia:

- **D8 (TrasladandoPacienteClinicaState.RetornoClinica, Severidad Media):** Confirmado en integración por **CC2**. Mientras que CM21 ubicó el método defectuoso con mocks, CC2 expuso su impacto en un flujo real: luego de trasladar a un paciente y regresar, la ambulancia queda erróneamente en RegresandoDelTallerState en lugar de DisponibleState.
- **D9 (Ambulancia.retornoAutomatico, Severidad Media):** Detectado por **CC4**. Cuando el retorno automático transiciona desde AtendiendoEnDomicilioState hacia RegresandoClinicaSinPacienteState, cambia el estado pero omite actualizar ocupado = false (ni llama a notifyAll()), violando el invariante de ocupación (S10) y dejando la ambulancia bloqueada como ocupada.
- **D10 (Ambulancia.atenderDomicilio, trasladarALaClinica y solicitarMantenimiento, Severidad Media):** Detectado por **CC5**. Los tres métodos de solicitud ejecutan this.ocupado = true y notifican éxito **antes** de delegar en el estado actual y sin verificar si la transición fue aceptada. Si la ambulancia está libre en un estado intermedio (como RegresandoDelTallerState) y recibe un pedido que ese estado rechaza, queda igualmente marcada como ocupada (isOcupado() == true), rompiendo el invariante de ocupación (S11).

- **D11 (RetornoAutomatico.run, Severidad Alta):** Detectado por **CC9**. La condición del bucle principal en el hilo RetornoAutomatico está codificada al revés: while (!this.ambulancia.isSimulacionActiva()). En consecuencia, cuando se inicia la simulación con simulacionActiva == true, el hilo sale inmediatamente del bucle while y muere sin realizar ningún retorno automático.
- **D12 (TrasladandoPacienteClinicaState.RetornoClinica, Severidad Media):** Detectado por **CC11**. Este estado ejecuta un Thread.sleep(1000) dentro de su método RetornoClinica(), el cual es invocado desde métodos synchronized de Ambulancia. Al dormir reteniendo el monitor del objeto compartido, congela durante 1 segundo a cualquier otro hilo que intente siquiera consultar isOcupado().

Falla sembrada (FC1) y observaciones de revisión de código:

- Para demostrar la capacidad de la suite ante omisiones de notificación de monitor sin alterar el código productivo, se implementó en el paquete de pruebas la subclase support/AmbulanciaWithoutNotifyAll, donde regresarSinPaciente() libera la bandera de ocupación pero no invoca notifyAll() (**FC1**). El caso **CC13** ejecuta el escenario de CC6 sobre dicho objeto y comprueba exitosamente que el hilo en espera jamás despierta.
- Por inspección de caja blanca se observó además que: (1) getEstadoActual() no es synchronized ni volatile, exponiendo un riesgo potencial de visibilidad de memoria entre hilos; (2) Clinica.iniciarSimulacion registra a cada asociado dos veces en la lista de la ambulancia; y (3) TrasladandoPacienteClinicaState.RetornoClinica captura Exception con un bloque vacío, tragándose posibles interrupciones del hilo (InterruptedException).

5.2.5. Conclusiones del Ítem 3 (Mocks y Concurrencia)

- La combinación de pruebas atómicas con mocks (Sección 5.1) y pruebas integradas con hilos reales (Sección 5.2) demostró ser altamente complementaria: el aislamiento con Mockito permitió verificar en milisegundos las 24 celdas de la tabla de estados y localizar con exactitud los defectos **D7** y **D8** en las clases de estado concretas tras confirmar que Ambulancia delegaba correctamente (CM1 a CM9).
- A su vez, acoplar la máquina de estados real y someterla a ejecución concurrente expuso cuatro defectos adicionales (**D9**, **D10**, **D11** y **D12**) que ningún mock podía revelar, dado que emergen de la interacción entre la bandera ocupado de Ambulancia y las transiciones de los estados reales (**D9**, **D10**), de la condición invertida en el hilo RetornoAutomatico (**D11**) y de la retención indebida del lock durante un Thread.sleep (**D12**).

6. Conclusiones

El desarrollo del presente trabajo práctico permitió diseñar, implementar y ejecutar una estrategia integral de validación sobre los subsistemas de persistencia y de simulación concurrente de la aplicación *Clínica*, estructurada en cuatro suites independientes que totalizaron 84 casos de prueba. A partir de la ejecución de dichas pruebas sobre el código provisto, se verificó el cumplimiento de los flujos principales (66 casos exitosos), pero fundamentalmente se logró exponer un total de 12 defectos en el producto —dos en tiempo de compilación y diez en tiempo de ejecución—, además de comprobar la sensibilidad de las pruebas ante la falla en concurrencia. Estos resultados demuestran empíricamente que el sistema entregado presenta fallas críticas que comprometen tanto la integridad de los datos persistidos como la operatividad de su simulación multihilo.

En lo que respecta al módulo de persistencia, la combinación de pruebas unitarias sobre AsociadoDTO y pruebas de integración contra un motor MySQL 8 real evidenció cómo pequeños errores de implementación impactan severamente en el sistema acoplado. Mientras que el análisis de valores límite a nivel atómico permitió detectar y corregir el rechazo indebido del valor frontera cero en el número de domicilio (**D3**), la ejecución sobre el motor de base de datos real destapó defectos que habrían permanecido ocultos bajo un entorno simulado: la imposibilidad de recuperar asociados de forma individual por omitir el avance del cursor del ResultSet (**D5**), la entrega de conexiones cerradas por parte del Singleton (**D4**) y la corrupción silenciosa del primer registro de la tabla durante las actualizaciones por fijar estáticamente el identificador en la sentencia SQL (**D6**).

Por su parte, la validación de la máquina de estados de Ambulancia puso de manifiesto la complementariedad entre el aislamiento mediante dobles de prueba y la evaluación integrada con hilos reales. El empleo de *mocks* con Mockito permitió auditar de manera determinista las 24 transiciones de la tabla de estados y localizar con exactitud en qué clases residían las omisiones de notificación a la vista (**D7**) y la transición errónea tras el traslado de un paciente (**D8**). Posteriormente, al acoplar los objetos reales y someterlos a contención concurrente, emergieron los problemas propios de la sincronización y del manejo de estado compartido: el desfasaje de la bandera de ocupación (**D9** y **D10**), la retención indebida del monitor durante esperas pasivas (**D12**) y la inoperancia total del hilo RetornoAutomatico debido a una condición de bucle invertida (**D11**). En suma, la experiencia confirma que derivar los oráculos estrictamente desde la especificación funcional y escalonar las pruebas desde el nivel atómico hacia la integración real constituye un enfoque eficaz para evaluar la calidad del software.

6.1. Resumen global de ejecución

La siguiente tabla consolida las métricas obtenidas en las cuatro suites implementadas (el detalle caso por caso se encuentra en el Apéndice A y las capturas del entorno en el Apéndice B):

Suite de prueba	Etiqueta (@Tag)	Casos totales	Pasan (Verde)	Fallan (Rojo)	Defectos productivos detectados	Fallas sembradas detectadas
1. Unitaria (AsociadoDTO)	unit	15	14	1	D3	—
2. Integración con MySQL	integration	23	17	6	D4, D5, D6 *(y reválida de D3)*	—
3A. Mocks (Ambulancia / State)	mock	33	27	6	D7, D8	—
3B. Integración y Concurrencia	concurrency	13	8	5	D8, D9, D10, D11, D12	FC1 (1 de 1)
Total general	—	**84**	**66**	**18**	**10 defectos en ejecución (+ D1 y D2)**	**1 de 1 (100%)**

A los 10 defectos detectados durante la ejecución dinámica de las suites se suman **D1** y **D2**, identificados en fase de compilación (mvn compile). En total se registraron **12 defectos**: **5 de severidad Alta** (D1, D2, D5, D6 y D11), **6 de severidad Media** (D3, D4, D8, D9, D10 y D12) y **1 de severidad Baja** (D7). Solo se corrigió en el repositorio el defecto **D3** por indicación expresa de la Consigna 2.

6.2. Registro maestro de defectos

Escala de severidad aplicada:

- **Alta:** Impide compilar o ejecutar una operación principal, pierde o corrompe datos, o anula un hilo crítico.
- **Media:** Produce un resultado o estado incorrecto que viola la especificación, con rodeo posible, o degrada el rendimiento concurrente.
- **Baja:** Inconsistencia u omisión de notificación secundaria sin impacto funcional inmediato.

ID	Componente afectado	Descripción del defecto y criterio violado	Detectado por	Esperado vs. Obtenido	Sev.	Estado final
D1	EnTallerState	Declara package model.modeloDominio.ambulancia pero reside en el directorio modelo/... (viola reglas de compilación Java y UML).	mvn compile	**Esp:** Compilación exitosa. **Obt:** Error duplicate class / cannot find symbol.	Alta	No se corrige en repo (arreglado en copia temporal, Apéndice C).

ID	Componente afectado	Descripción del defecto y criterio violado	Detectado por	Esperado vs. Obtenido	Sev.	Estado final
D2	TrasladandoPacienteClinicaState	La clase pública se encuentra en el archivo con nombre erróneo TrasladandoPacienteClinicasState.java (con "s" extra).	mvn compile	**Esp:** Compilación exitosa. **Obt:** Error de compilación por nombre de archivo.	Alta	No se corrige en repo (arreglado en copia temporal, Apéndice C).
D3	AsociadoDTO.setNumero	Valida assert numero > 0 en lugar de numero >= 0, rechazando el valor límite válido 0 (viola SRS 3.1).	CU4, CI21	**Esp:** setNumero(0) asigna 0. **Obt:** Lanza AssertionError.	Media	**Corregido** (6cda0de) por pedido de la Consigna 2.
D4	BaseDeDatos.getInstancia	Tras llamar a cerrarConexion(), sigue devolviendo la misma instancia con su conexión cerrada (viola SRS 3.2 y S6).	CI6	**Esp:** Conexión abierta y activa. **Obt:** isClosed() == true.	Media	No se corrige (evidencia de detección en testing).
D5	DataAccessObject.cargar	Intenta leer columnas del ResultSet sin invocar previamente rs.next() (viola SRS 3.3 de carga individual por ID).	CI9, CI10, CI11	**Esp:** DTO de la fila pedida o null. **Obt:** SQLException: Before start of result set.	Alta	No se corrige (evidencia de detección en testing).
D6	BaseDeDatos.ejecutarActualizacion	El UPDATE fija setInt(8, 1) (WHERE id = 1), modificando siempre la fila 1 en vez de buscar por DNI (viola SRS 3.3).	CI16, CI17	**Esp:** Actualiza solo la fila del DNI. **Obt:** Duplicate entry (CI16) o pisa fila 1 (CI17).	Alta	No se corrige (evidencia de detección en testing).
D7	4 estados (SolicitudDeAtencionDomicilio) y EnTallerState.RetornoClinica	Al rechazar un pedido no avisan a los observadores porque notifyObservers(...) está comentado (viola Informe de Diseño).	CM15, CM19, CM23, CM25, CM27	**Esp:** Invoca notifyObservers(...). **Obt:** Sin interacción con el mock.	Baja	No se corrige (evidencia de detección en testing).

ID	Componente afectado	Descripción del defecto y criterio violado	Detectado por	Esperado vs. Obtenido	Sev.	Estado final
D8	TrasladandoPacienteClinicaState.RetornoClinica	Transiciona hacia RegresandoDelTallerState en lugar de DisponibleState (viola Javadoc y diagrama UML).	CM21, CC2	**Esp:** DisponibleState. **Obt:** RegresandoDelTallerState.	Media	No se corrige (evidencia de detección en testing).
D9	Ambulancia.retornoAutomatico	Al pasar de AtendiendoEnDomicilio a RegresandoClinicaSinPaciente cambia el estado pero no libera ocupado (viola invariante y S10).	CC4	**Esp:** isOcupado() == false. **Obt:** isOcupado() == true.	Media	No se corrige (evidencia de detección en testing).
D10	Ambulancia (atenderDomicilio, trasladar..., solicitar...)	Marca ocupado = true y notifica éxito antes de delegar, aún cuando el estado actual rechace el pedido (viola invariante y S11).	CC5	**Esp:** Libre en RegresandoDelTaller. **Obt:** Queda marcada ocupada.	Media	No se corrige (evidencia de detección en testing).
D11	RetornoAutomatico.run	Condición del bucle invertida (while (!isSimulacionActiva())): con la simulación activa el hilo termina de inmediato (viola Javadoc).	CC9	**Esp:** Retorna a Disponible en < 15 s. **Obt:** El hilo muere al arrancar.	Alta	No se corrige (evidencia de detección en testing).
D12	TrasladandoPacienteClinicaState.RetornoClinica	Ejecuta Thread.sleep(1000) dentro de un método synchronized de Ambulancia, bloqueando a los demás hilos (viola Informe y S12).	CC11	**Esp:** isOcupado() responde sin bloqueo. **Obt:** Espera a que termine el sleep.	Media	No se corrige (evidencia de detección en testing).

6.3. Evaluación metodológica y limitaciones

Puntos clave validados por la estrategia:

- **Independencia del oráculo:** Tomar el resultado esperado desde la documentación (SRS, UML e Informe de Diseño) y nunca desde el código fue determinante para evitar falsos verdes en casos como CC2 o CI17.
- **Reclasificación de validaciones vs. precondiciones:** Distinguir las reglas verificadas activamente con assert en los setters de AsociadoDTO y en DAO.guardar

respecto de las precondiciones puras permitió incorporar las clases inválidas (CU9 a CU15 y CI23) y verificar ambos lados de la frontera numérica (-1 y 0).

- **Canales independientes y esperas observables:** La verificación SQL externa con datos distintos por columna destapó la sobrescritura silenciosa de **D6**, mientras que la espera de estados observables de los hilos (WAITING, TIMED_WAITING) eliminó la dependencia del reloj del sistema en las pruebas de concurrencia.

Limitaciones del estudio:

- **Condiciones de carrera latentes:** Que una suite concurrente resulte exitosa en los escenarios probados no demuestra formalmente la ausencia total de condiciones de carrera; la falta de volatile o synchronized en getEstadoActual() y en BaseDeDatos.getInstancia() quedó registrada por inspección estática de código.
- **Dependencia de -ea y de supuestos:** Las validaciones mediante assert requieren obligatoriamente ejecutar la JVM con la opción -ea habilitada. Asimismo, ante vacíos de la especificación fue necesario adoptar los supuestos **S1** a **S16** (de los cuales dependen casos como CI6, CM25, CC5 y CC11).
- **No modificación del código productivo:** Salvo para **D3**, las restricciones del trabajo impidieron aplicar las correcciones en el código de producción para verificar empíricamente que los 18 casos fallidos pasen a estado verde.

Apéndice A: Resultados detallados de ejecución

Las siguientes tablas provienen de los reportes generados por Maven Surefire (mvn clean test -Dgroups=<etiqueta>) conforme al procedimiento del Apéndice C, y se corresponden con las capturas de pantalla del Apéndice B.

A.1. Pruebas unitarias de AsociadoDTO (@Tag("unit"))

Ejecución sobre el AsociadoDTO original entregado (copia clinica-unit), Java 17.0.21.0.2. Total: 15 casos | 14 pasan | 1 falla.

ID	Clase de prueba	Método de prueba (@Test)	Resultado	Defecto	Motivo de la falla (Reporte Surefire)
CU1	AsociadoDTOTest	cu1_defaultConstructorLeavesDefaultValues	✅ Pasa	—	—
CU2	AsociadoDTOTest	cu2_constructorAssignsValidData	✅ Pasa	—	—
CU3	AsociadoDTOTest	cu3_stringSettersAssignValidText	✅ Pasa	—	—
CU4	AsociadoDTOTest	cu4_setNumeroAcceptsZero	❌ Falla	**D3**	AssertionError: setNumero(0) rechaza el valor límite 0.
CU5	AsociadoDTOTest	cu5_setNumeroAcceptsOne	✅ Pasa	—	—
CU6	AsociadoDTOTest	cu6_setNumeroAcceptsMaxValue	✅ Pasa	—	—
CU7	AsociadoDTOTest	cu7_stringSettersAcceptEmptyText	✅ Pasa	—	—
CU8	AsociadoDTOTest	cu8_stringSettersAcceptBlankText	✅ Pasa	—	—
CU9	AsociadoDTOTest	cu9_setNombreRejectsNull	✅ Pasa	—	—
CU10	AsociadoDTOTest	cu10_setApellidoRejectsNull	✅ Pasa	—	—
CU11	AsociadoDTOTest	cu11_setDniRejectsNull	✅ Pasa	—	—
CU12	AsociadoDTOTest	cu12_setCiudadRejectsNull	✅ Pasa	—	—
CU13	AsociadoDTOTest	cu13_setCalleRejectsNull	✅ Pasa	—	—
CU14	AsociadoDTOTest	cu14_setTelefonoRejectsNull	✅ Pasa	—	—
CU15	AsociadoDTOTest	cu15_setNumeroRejectsMinusOne	✅ Pasa	—	—

A.2. Pruebas de integración con MySQL (@Tag("integration"))

Ejecución sobre el código con D3 corregido (copia clinica-resto), Java 17.0.21.0.2. Total: 23 casos | 17 pasan | 6 fallan.

ID	Clase de prueba	Método de prueba (@Test)	Resultado	Defecto	Motivo de la falla (Reporte Surefire)
CI1	DatabaseIntegrationTest	ci1_getInstanciaReturnsSameInstance	✅ Pasa	—	—
CI2	DatabaseIntegrationTest	ci2_iniciarBDCreatesSchema	✅ Pasa	—	—
CI3	DatabaseIntegrationTest	ci3_iniciarBDCreatesTableWithColumns	✅ Pasa	—	—
CI4	DatabaseIntegrationTest	ci4_iniciarBDLoadsValidSeedData	✅ Pasa	—	—
CI5	DatabaseIntegrationTest	ci5_iniciarBDRecreatesTable	✅ Pasa	—	—
CI6	DatabaseIntegrationTest	ci6_getInstanciaAfterCerrarConexionGivesActiveConnection	❌ Falla	**D4**	getInstancia() devolvió un manejador con la conexión cerrada (isClosed() = true).
CI7	DataAccessObjectIntegrationTest	ci7_guardarInsertsEveryColumn	✅ Pasa	—	—
CI8	DataAccessObjectIntegrationTest	ci8_guardarRejectsDuplicateDni	✅ Pasa	—	—
CI9	DataAccessObjectIntegrationTest	ci9_cargarFirstRow	❌ Falla	**D5**	SQLException: Before start of result set.
CI10	DataAccessObjectIntegrationTest	ci10_cargarLastRow	❌ Falla	**D5**	SQLException: Before start of result set.
CI11	DataAccessObjectIntegrationTest	ci11_cargarMissingIdReturnsNull	❌ Falla	**D5**	SQLException: Illegal operation on empty result set.
CI12	DataAccessObjectIntegrationTest	ci12_cargarConLimiteOne	✅ Pasa	—	—

ID	Clase de prueba	Método de prueba (@Test)	Resultado	Defecto	Motivo de la falla (Reporte Surefire)
CI13	DataAccessObjectIntegrationTest	ci13_cargarConLimiteEqualToRows	✅ Pasa	—	—
CI14	DataAccessObjectIntegrationTest	ci14_cargarConLimiteGreaterThanRows	✅ Pasa	—	—
CI15	DataAccessObjectIntegrationTest	ci15_cargarBDReturnsWholeTable	✅ Pasa	—	—
CI16	DataAccessObjectIntegrationTest	ci16_actualizarChangesOnlyThatDni	❌ Falla	**D6**	SQLIntegrityConstraintViolationException: Duplicate entry '22334455' for key 'Asociados.DNI'.
CI17	DataAccessObjectIntegrationTest	ci17_actualizarMissingDniChangesNothing	❌ Falla	**D6**	La tabla cambió: la fila 1 quedó sobrescrita con los datos del asociado.
CI18	DataAccessObjectIntegrationTest	ci18_eliminarExistingDni	✅ Pasa	—	—
CI19	DataAccessObjectIntegrationTest	ci19_eliminarMissingDniChangesNothing	✅ Pasa	—	—
CI20	DataAccessObjectIntegrationTest	ci20_eliminarTreatsSqlInjectionAsPlainValue	✅ Pasa	—	—
CI21	DataAccessObjectIntegrationTest	ci21_cargarBDMapsNumeroZero	✅ Pasa	—	—
CI22	DataAccessObjectIntegrationTest	ci22_cargarBDWithoutTableThrowsSQLException	✅ Pasa	—	—
CI23	DataAccessObjectIntegrationTest	ci23_guardarRejectsNull	✅ Pasa	—	—

A.3. Pruebas con Mocks (@Tag("mock"))

Ejecución sobre el código con D3 corregido (copia clinica-resto), Java 17.0.21.0.2. Total: 33 casos | 27 pasan | 6 fallan.

ID	Clase de prueba	Método de prueba (@Test)	Resultado	Defecto	Motivo de la falla (Reporte Surefire)
CM1	AmbulanciaMockTest	cm1_atenderDomicilioDelegatesToState	✅ Pasa	—	—
CM2	AmbulanciaMockTest	cm2_trasladarDelegatesToState	✅ Pasa	—	—
CM3	AmbulanciaMockTest	cm3_solicitarMantenimientoDelegatesToState	✅ Pasa	—	—
CM4	AmbulanciaMockTest	cm4_regresarSinPacienteDelegatesAndReleases	✅ Pasa	—	—
CM5	AmbulanciaMockTest	cm5_volviendoDelTallerDelegatesAndReleases	✅ Pasa	—	—
CM6	AmbulanciaMockTest	cm6_retornoAutomaticoDelegatesToState	✅ Pasa	—	—
CM7	AmbulanciaMockTest	cm7_atenderDomicilioAfterSimulacionDoesNothing	✅ Pasa	—	—
CM8	AmbulanciaMockTest	cm8_trasladarWhenBusyAfterSimulacionDoesNotWait	✅ Pasa	—	—
CM9	AmbulanciaMockTest	cm9_delegatesToStateSetByPreviousTransition	✅ Pasa	—	—
CM10	StatesMockTest	cm10_disponibleTraslado	✅ Pasa	—	—
CM11	StatesMockTest	cm11_disponibleAtencionDomicilio	✅ Pasa	—	—
CM12	StatesMockTest	cm12_disponibleMantenimiento	✅ Pasa	—	—
CM13	StatesMockTest	cm13_disponibleRetornoClinica	✅ Pasa	—	—
CM14	StatesMockTest	cm14_atendiendoTraslado	✅ Pasa	—	—
CM15	StatesMockTest	cm15_atendiendoAtencionDomicilio	❌ Falla	**D7**	Wanted but not invoked: ambulancia.notifyObservers(...).
CM16	StatesMockTest	cm16_atendiendoMantenimiento	✅ Pasa	—	—
CM17	StatesMockTest	cm17_atendiendoRetornoClinica	✅ Pasa	—	—
CM18	StatesMockTest	cm18_trasladandoTraslado	✅ Pasa	—	—

ID	Clase de prueba	Método de prueba (@Test)	Resultado	Defecto	Motivo de la falla (Reporte Surefire)
CM19	StatesMockTest	cm19_trasladandoAtencionDomicilio	❌ Falla	**D7**	Wanted but not invoked: ambulancia.notifyObservers(...).
CM20	StatesMockTest	cm20_trasladandoMantenimiento	✅ Pasa	—	—
CM21	StatesMockTest	cm21_trasladandoRetornoClinica	❌ Falla	**D8**	Argument(s) are different: se esperaba setState(DisponibleState) y se llamó con RegresandoDelTallerState.
CM22	StatesMockTest	cm22_enTallerTraslado	✅ Pasa	—	—
CM23	StatesMockTest	cm23_enTallerAtencionDomicilio	❌ Falla	**D7**	Wanted but not invoked: ambulancia.notifyObservers(...).
CM24	StatesMockTest	cm24_enTallerMantenimiento	✅ Pasa	—	—
CM25	StatesMockTest	cm25_enTallerRetornoClinica	❌ Falla	**D7**	Wanted but not invoked: ambulancia.notifyObservers(...).
CM26	StatesMockTest	cm26_regresandoDelTallerTraslado	✅ Pasa	—	—
CM27	StatesMockTest	cm27_regresandoDelTallerAtencionDomicilio	❌ Falla	**D7**	Wanted but not invoked: ambulancia.notifyObservers(...).
CM28	StatesMockTest	cm28_regresandoDelTallerMantenimiento	✅ Pasa	—	—
CM29	StatesMockTest	cm29_regresandoDelTallerRetornoClinica	✅ Pasa	—	—
CM30	StatesMockTest	cm30_regresandoSinPacienteTraslado	✅ Pasa	—	—
CM31	StatesMockTest	cm31_regresandoSinPacienteAtencionDomicilio	✅ Pasa	—	—

ID	Clase de prueba	Método de prueba (@Test)	Resultado	Defecto	Motivo de la falla (Reporte Surefire)
CM32	StatesMockTest	cm32_regresandoSinPacienteMantenimiento	✅ Pasa	—	—
CM33	StatesMockTest	cm33_regresandoSinPacienteRetornoClinica	✅ Pasa	—	—

A.4. Pruebas de integración y concurrencia (@Tag("concurrency"))

Ejecución sobre el código con D3 corregido (copia clinica-resto), Java 17.0.21.0.2. Total: 13 casos | 8 pasan | 5 fallan.

ID	Clase de prueba	Método de prueba (@Test)	Resultado	Defecto	Motivo de la falla (Reporte Surefire)
CC1	AmbulanciaStateMachineTest	cc1_atencionDomicilioFlow	✅ Pasa	—	—
CC2	AmbulanciaStateMachineTest	cc2_trasladoFlow	❌ Falla	**D8**	Estado esperado DisponibleState, obtenido RegresandoDelTallerState.
CC3	AmbulanciaStateMachineTest	cc3_mantenimientoFlow	✅ Pasa	—	—
CC4	AmbulanciaStateMachineTest	cc4_retornoAutomaticoKeepsOccupancyInvariant	❌ Falla	**D9**	Ocupación incorrecta en RegresandoClinicaSinPacienteState: esperado libre, obtenido ocupada.

ID	Clase de prueba	Método de prueba (@Test)	Resultado	Defecto	Motivo de la falla (Reporte Surefire)
CC5	AmbulanciaStateMachineTest	cc5_rejectedRequestDoesNotOccupy	❌ Falla	**D10**	Ocupación incorrecta en RegresandoDelTallerState: esperado libre, obtenido ocupada.
CC6	AmbulanciaConcurrencyTest	cc6_waitingRequestWakesUpOnRegresarSinPaciente	✅ Pasa	—	—
CC7	AmbulanciaConcurrencyTest	cc7_waitingRequestWakesUpOnVolviendoDelTaller	✅ Pasa	—	—
CC8	AmbulanciaConcurrencyTest	cc8_endingSimulacionReleasesAllWaitingThreads	✅ Pasa	—	—
CC9	AmbulanciaConcurrencyTest	cc9_retornoAutomaticoReturnsAmbulance	❌ Falla	**D11**	No ocurrió en 15000 ms: el retorno automático no devolvió la ambulancia a Disponible.
CC10	AmbulanciaConcurrencyTest	cc10_retornoAutomaticoAfterSimulacionDoesNothing	✅ Pasa	—	—
CC11	AmbulanciaConcurrencyTest	cc11_slowTransitionDoesNotBlockQueries	❌ Falla	**D12**	isOcupado() tuvo que esperar a que terminara el retorno a la clínica.
CC12	AmbulanciaConcurrencyTest	cc12_smallSimulacionFinishes	✅ Pasa	—	—
CC13	AmbulanciaConcurrencyTest	cc13_seededMissingNotifyAllIsDetected	✅ Pasa	**FC1** *(Sembrada)*	—

Apéndice B: Capturas de ejecución en el IDE

Cada figura exhibe el panel de resultados de JUnit en IntelliJ IDEA correspondiente a la ejecución de una suite: los casos exitosos se visualizan en verde y los fallidos en rojo junto con la traza de la excepción o aserción que demuestra la detección del defecto.

B.1. Pruebas unitarias de AsociadoDTO

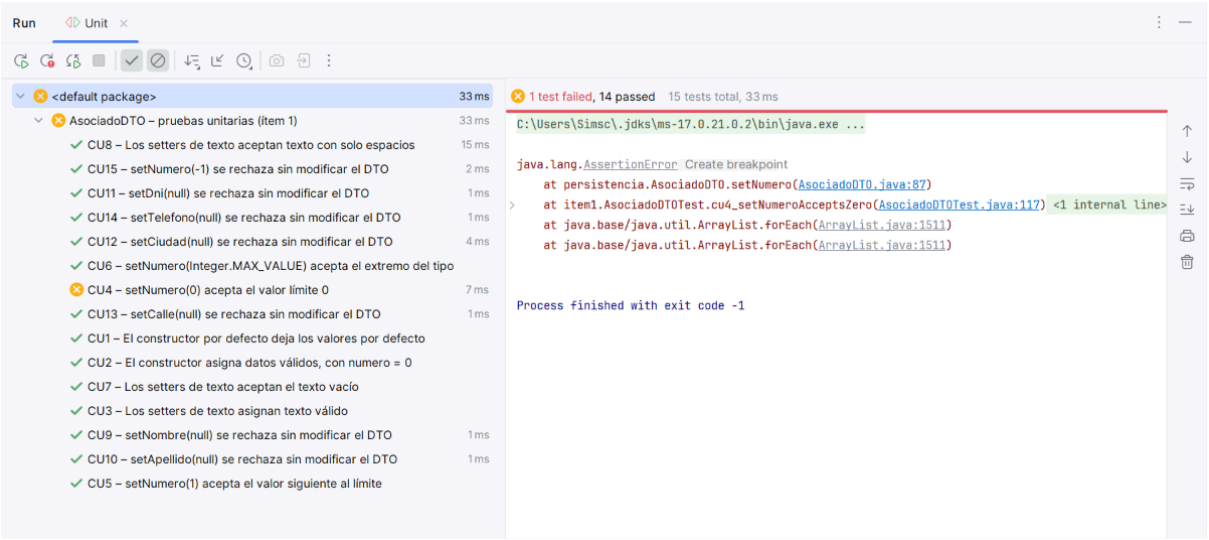


Figura B.1. Suite unitaria (unit): 15 casos ejecutados, 14 pasan y falla CU4 (defecto **D3**).

B.2. Pruebas de integración con MySQL

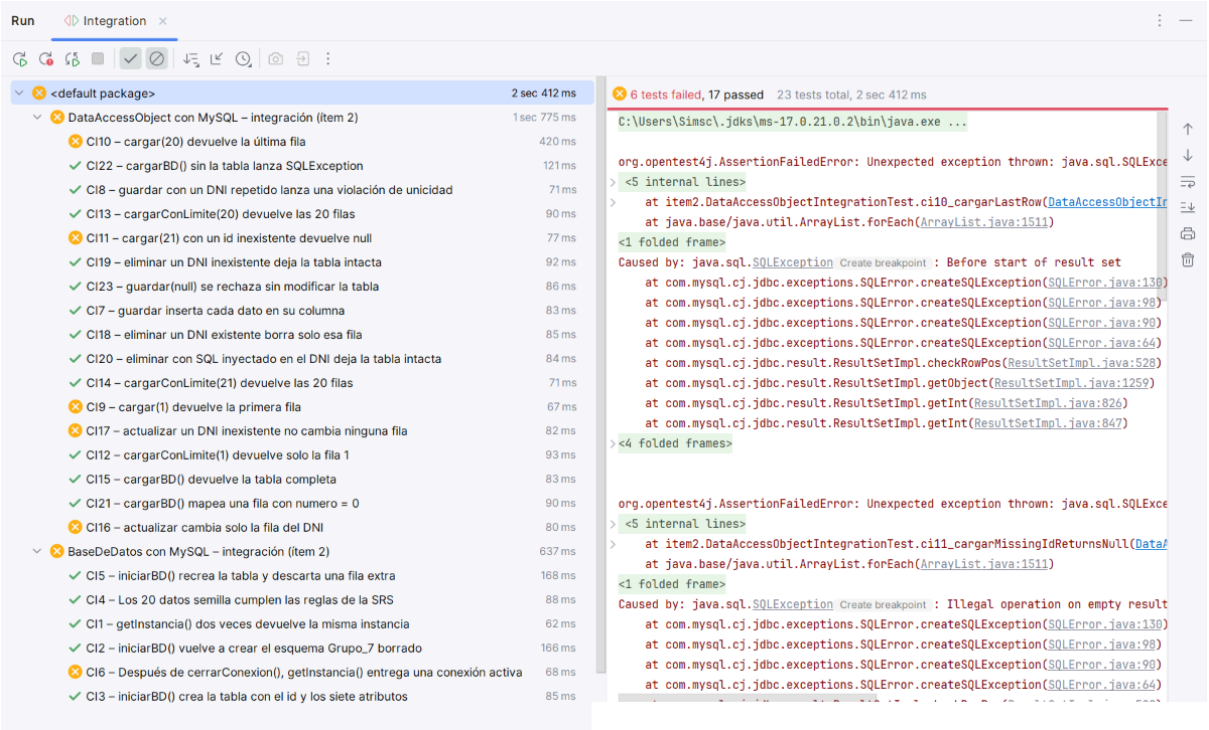


Figura B.2. Suite de integración (integration): 23 casos ejecutados, 17 pasan y fallan CI6 (**D4**), CI9, CI10 y CI11 (**D5**), y CI16 y CI17 (**D6**).

B.3. Pruebas con Mocks

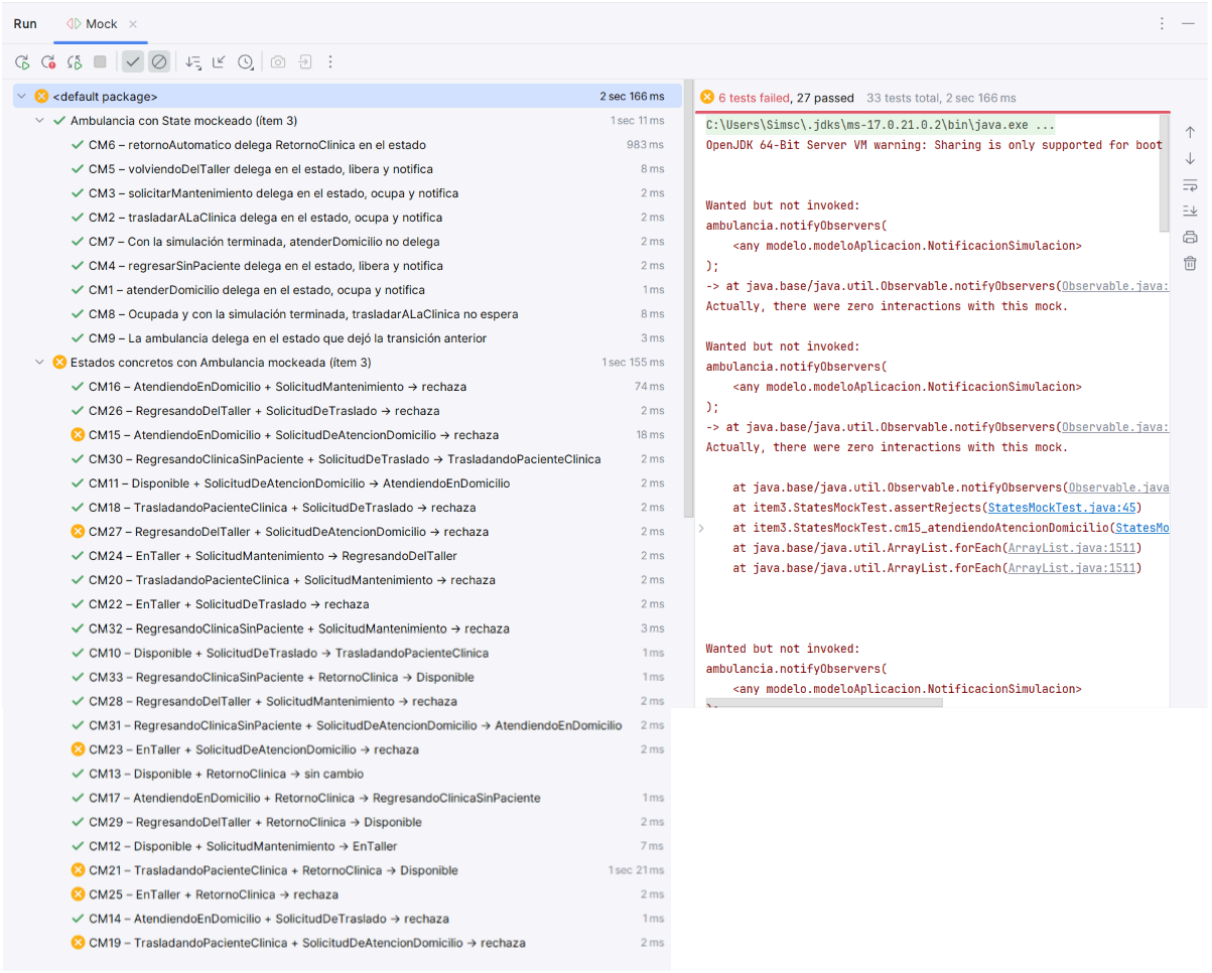


Figura B.3. Suite de mocks (mock): 33 casos ejecutados, 27 pasan y fallan CM15, CM19, CM23, CM25 y CM27 (**D7**), y CM21 (**D8**).

B.4. Pruebas de integración y concurrencia

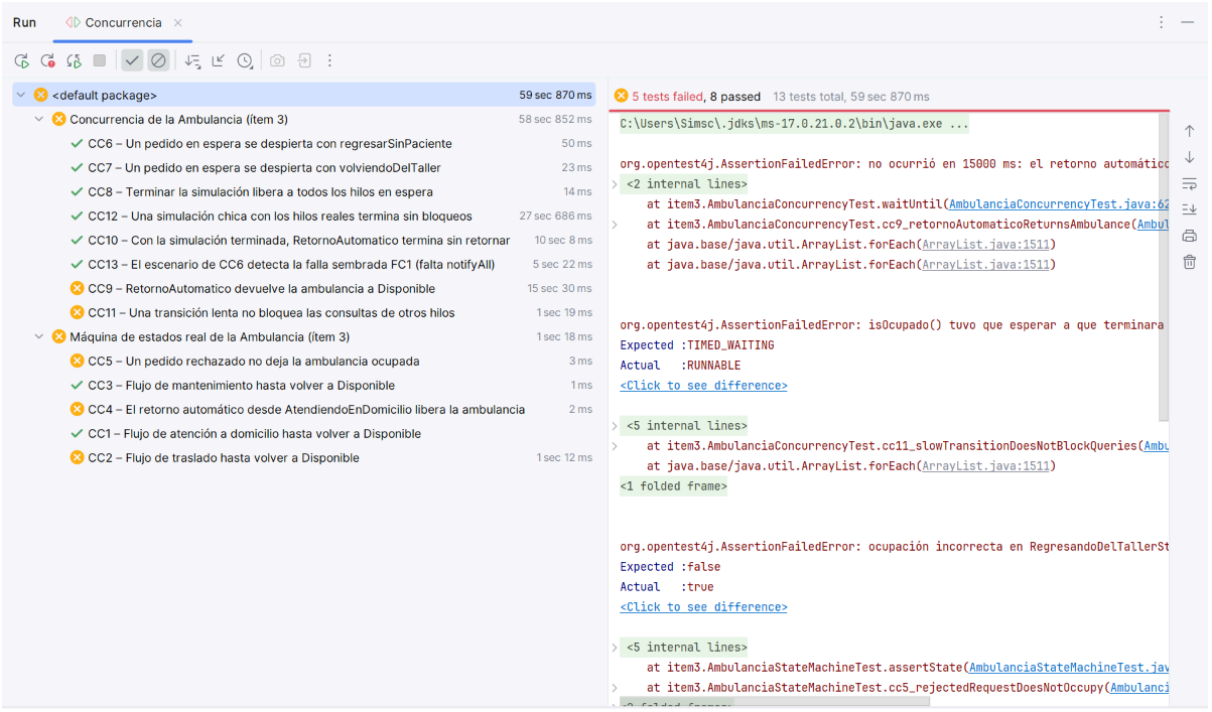


Figura B.4. Suite de concurrencia (concurrency): 13 casos ejecutados, 8 pasan y fallan CC2 (**D8**), CC4 (**D9**), CC5 (**D10**), CC9 (**D11**) y CC11 (**D12**).

Apéndice C: Entorno de ejecución y reproducción

C.1. Ajustes del entorno local

Siguiendo el principio de no alterar el código bajo prueba, el entorno de ejecución fue adaptado a las configuraciones codificadas de manera estática en la aplicación:

1. **Autenticación en MySQL 8.0:** La clase BaseDeDatos define de forma fija la cadena de conexión hacia localhost:3306 con el usuario progra_c, contraseña progra_c, parámetro useSSL=false y sin allowPublicKeyRetrieval=true. Dado que el plugin por defecto de MySQL 8 (caching_sha2_password) rechaza dicha conexión y BaseDeDatos captura la SQLException sin inicializar el atributo conexion, el usuario local debe crearse utilizando el plugin mysql_native_password y con permisos globales para permitir la creación y borrado del esquema Grupo_7:

```SQL
CREATE USER 'progra_c'@'localhost' IDENTIFIED WITH mysql_native_password BY 'progra_c';
GRANT ALL PRIVILEGES ON *.* TO 'progra_c'@'localhost';
```

2. **Copias temporales de compilación (git worktree):** Dado que los defectos **D1** y **D2** impiden compilar el proyecto pero no deben corregirse en la rama principal del repositorio (y que la suite unitaria debe correrse antes de la corrección de **D3**, mientras que las restantes deben correrse con **D3** corregido), se generan dos árboles de trabajo temporales fuera del repositorio:

```Shell
git worktree add --detach ../clinica-unit master
git worktree add --detach ../clinica-resto master

cd ../clinica-unit
git cherry-pick -n c1ff2a6   # Aplica arreglo de compilación D1 y D2 solo en la copia
git revert -n 6cda0de        # Restaura AsociadoDTO original sin corregir D3

cd ../clinica-resto
git cherry-pick -n c1ff2a6   # Aplica arreglo de compilación D1 y D2 con D3 ya corregido
```

C.2. Comandos de ejecución por suite

Suite de prueba	Directorio de trabajo	Etiqueta JUnit (@Tag)	Comando Maven equivalente
Unitaria	../clinica-unit	unit	mvn clean test -Dgroups=unit -Dmaven.test.failure.ignore=true
Integración con MySQL	../clinica-resto	integration	mvn clean test -Dgroups=integration -Dmaven.test.failure.ignore=true
Mocks	../clinica-resto	mock	mvn clean test -Dgroups=mock -Dmaven.test.failure.ignore=true
Concurrencia	../clinica-resto	concurrency	mvn clean test -Dgroups=concurrency -Dmaven.test.failure.ignore=true

Tanto en el plugin Surefire de Maven como en las configuraciones de ejecución de IntelliJ IDEA se incluye obligatoriamente el argumento de máquina virtual -ea (*enable assertions*) para activar las instrucciones assert del código productivo.

Apéndice D: Referencias y trazabilidad con la SRS

A continuación se transcriben los requisitos de la Especificación de Requisitos de Software del módulo de persistencia (docs/SRS_Persistencia_Clinica.pdf) junto con su trazabilidad directa hacia los casos de prueba y defectos documentados en este informe:

- **SRS 2.2 — Funciones del Sistema:** Encapsular atributos del asociado con validaciones de integridad; administrar un ciclo de vida centralizado de conexión con instancia única; proveer y poblar automáticamente el esquema al iniciar; y ejecutar operaciones CRUD mediante consultas SQL parametrizadas. *(Provee el marco general de validación de las Consignas 1 y 2)*.
- **SRS 3.1 — Requisitos de Transferencia de Datos e Integridad (AsociadoDTO):** Establece que nombre, apellido, dni (identificador único), ciudad, calle y telefono son de tipo String y *"No pueden ser nulos"*, mientras que numero es de tipo int y *"Debe ser mayor o igual a 0"*.
    - *Trazabilidad en Ítem 1:* Oráculo de las 14 clases válidas (V1–V14), las 7 clases inválidas (I1–I7), los casos CU1–CU15 y el defecto **D3** (CU4).
    - *Trazabilidad en Ítem 2:* Respalda la prueba de unicidad de DNI en la semilla (CI4) y ante inserciones duplicadas (CI8), así como la prueba de mapeo del límite numero = 0 (CI21).
- **SRS 3.2 — Requisitos de Conexión y Administración de Base de Datos (BaseDeDatos):**
    - *R3.2-a (Patrón Singleton con manejador activo único):* Evaluado en CI1 (misma instancia) y CI6 (reconexión tras cierre, supuesto **S6** y defecto **D4**).
    - *R3.2-b (Parámetros predefinidos de URL y credenciales):* Respetado mediante la adaptación del motor MySQL local descrita en el Apéndice C.
    - *R3.2-c (Rutina automática iniciarBD para esquema, tabla y semilla):* Evaluado en CI2 (creación de esquema), CI3 (columnas de tabla), CI4 (20 registros semilla válidos) y CI5 (recreación limpia de tabla).
    - *R3.2-d (Consultas parametrizadas con PreparedStatement contra inyección SQL):* Ejercitado en todas las pruebas del DAO (CI7–CI23) y verificado específicamente ante inyección SQL en CI20.
- **SRS 3.3 — Requisitos de Operaciones de Acceso a Datos (DataAccessObject):**
    - *R3.3-a (Operación de Guardado con validación de no nulidad, conversión a DTO e inserción):* Evaluado en CI7 (inserción completa), CI8 (rechazo de DNI repetido) y CI23 (rechazo de guardar(null)).
    - *R3.3-b (Operaciones de Carga individual por ID, por lote acotado y colección completa):* Evaluado en CI9–CI11 (carga individual, defecto **D5**), CI12–CI14 (carga con límite) y CI15, CI21, CI22 (carga completa).

- *R3.3-c (Operaciones de Actualización y Eliminación por criterios únicos como el DNI):* Evaluado en CI16–CI17 (actualización por DNI, defecto **D6**) y CI18–CI20 (eliminación por DNI).